Alma Mater Studiorum – Università di Bologna

Department of Electrical, Electronic, and Information Engineering
(DEI)

Project Report

Adaptive-DC-Motor-Control

Student: Abess Ouardi

Date: January 2025

Bologna, Italy

Contents

Chapter 1

Introduction

The Laboratory of Automation Systems provided an in-depth exploration of advanced control systems and their practical applications. A central focus was the implementation of enhanced control algorithms on embedded processors, making it possible to analyze and compare their performance across different methods.

The sessions also delved into the core components of an inverter, including Pulse Width Modulation (PWM), current sensing, and error correction codes, essential for efficient operation. Additionally, the use of encoders for precise feedback enhanced the understanding of system dynamics and accuracy. Automatic code generation was another critical aspect, showcasing how it simplifies development and accelerates the design process.

This report presents the key methodologies and outcomes of the laboratory.

Chapter 2

Hardware Setup

Overview of our motor setup

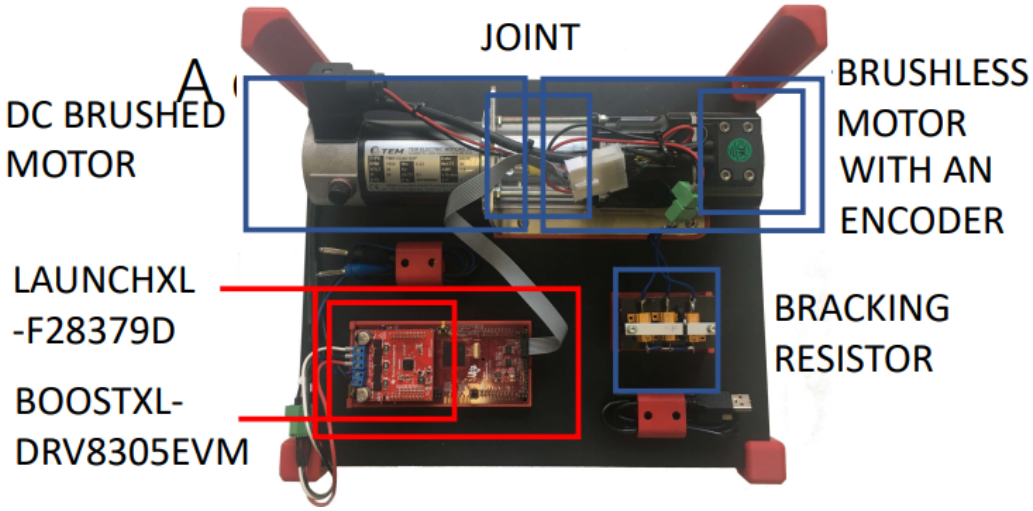

Figure 2.1: Motor setup

2.1 LAUNCHXL-F28379D

It's a development kit from Texas Instruments; it's part of the C2000 LaunchPad series and is designed for control-oriented applications.
Its key features:

- **Microcontroller**:
  Equipped with the TMS320F28379D, a high-performance 32-bit microcontroller
- **Dual-Core Architecture**:
  It has dual-core CPUs that allow parallel execution of tasks, which is particularly useful in real-time system.

- **High-resolution PWM**:
  Supports high-resolution PWM for precise control in power electronics and motor drives,
- **communication and connectivity**:
  includes various communication protocols, such as: CAN (Control Area Network), SPI (Serial Peripheral Interface), I2C, UART,Ethernet. Then it has USB connectivity for programming and communication.
- **Programming and debugging**:
  Integrated XDS110 Debug probe for on-chip debugging and programming.
- **Analog Capabilities**:
  Multiple ADC for data acquisition in real-time control applications.
- **Expansion Options**:
  BoosterPack headers for adding external modules or custom circuits

Basically it's a board that serves as the main controller, providing computational power, real-time control and peripheral interfacing.

2.2 BOOSTXL-DRV8305EVM

It's a motor driver designed for high-efficiency motor control with protection features and flexibility. Its key features include:

- **3-phase gate driver**: controls the high-side and low-side MOSFETs for BLCD motors.
- **Integrated Current Sensing**: Supports precise motor current measurements.
- **Fault Protection**: Overcurrent, overvoltage and thermal protection to ensure safety during operation.
- **Voltage range**: Operates within 4.4V to 45V, suitable for most BLDC motor applications.

The first task in the laboratory was to derive a mathematical model of the motor. This step was crucial as it allowed simulations to be run, providing insight into the motor's dynamic behavior and enabling the testing of control strategies in a virtual environment before applying them to the physical system.

Chapter 3

DC Motor Modeling

The equations and derivations represent the mathematical model of a DC motor, describing its electrical and mechanical dynamics. These equations are converted into the Laplace domain to derive transfer functions.

3.1 Electrical Dynamics of the DC Motor

The electrical circuit is modeled using Kirchhoff's Voltage Law (KVL):

$$V_A(s) = R_A I_A(s) + sL_A I_A(s) + V_M(s) \tag{3.1}$$

where:

- $V_A(s)$: Applied armature voltage (Laplace domain),
- $I_A(s)$: Armature current,
- R_A: Armature resistance,
- L_A: Armature inductance,
- $V_M(s)$: Back electromotive force (back EMF).

The back EMF is given by:

$$V_M(s) = k_m \Omega_m(s) \tag{3.2}$$

where:

- k_m: Motor constant (back EMF constant),
- $\Omega_m(s)$: Angular velocity of the motor.

3.2 Mechanical Dynamics of the DC Motor

The rotational motion is modeled using Newton's second law for rotation:

$$C_m(s) = k_m I_A(s) \tag{3.3}$$

where:

- $C_m(s)$: Electromagnetic torque,
- k_m: Motor constant (torque constant).

The relationship between torque and angular velocity is:

$$sJ\Omega_m(s) = C_m(s) - b\Omega_m(s) \tag{3.4}$$

where:

- J: Moment of inertia of the rotor,
- b: Viscous friction coefficient,
- $C_m(s)$: Electromagnetic torque.

Transfer Functions

From the equations above, the transfer functions can be derived:

1. **Armature Current Transfer Function**:

$$I_A(s) = \frac{V_A(s) - V_M(s)}{R_A + sL_A} \tag{3.5}$$

2. **Mechanical Transfer Function**:

$$\frac{\Omega_m(s)}{C_m(s)} = \frac{1}{b + sJ} \tag{3.6}$$

3.3 Complete DC Motor Model

By substituting $C_m(s) = k_m I_A(s)$ and $V_M(s) = k_m \Omega_m(s)$, the closed-loop transfer function for the motor is:

$$\Omega_m(s) = \frac{(V_A(s) - k_m \Omega_m(s))}{R_A + sL_A} \cdot k_m \cdot \frac{1}{b + sJ} \tag{3.7}$$

The parameters in the scheme are taken from the datasheet of the motor (Fig.3.2).

In particular, the following parameters were used in the modeling of this machine:

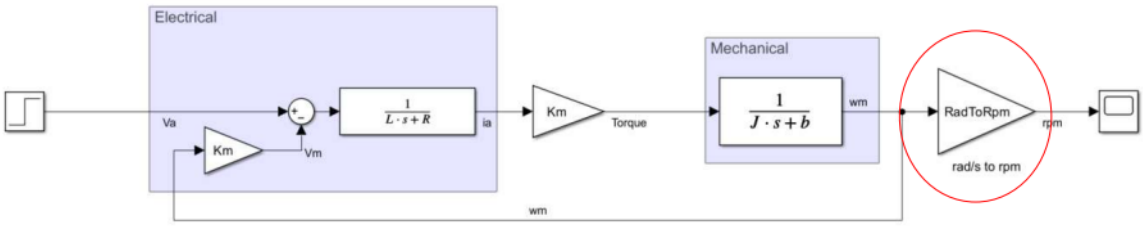

Figure 3.1: Motor Block scheme implemented in Simulink

- Inductance (L);
- Resistance (Ra);
- Torque constant (Km);
- Rotor inertia (J);
- Friction coefficient (b).

As evident from the motor's datasheet, the friction coefficient is not directly listed. This omission is because friction is an indirect property influenced by various factors, such as installation specifics, environmental conditions, motor usage, and the effects of aging and wear. Consequently, the friction coefficient varies across different applications. However, the impact of friction is indirectly reflected in the datasheet: the difference between the no-load speed and the rated speed represents the torque required to overcome both friction and the load. Thus, determining the friction coefficient requires dedicated calculations or tests. In the first Simulink model implemented (Fig. 3.3), the friction coefficient b was initially set to zero, and the results obtained under this assumption are presented in Fig. 5.

DATA TEST: 15.01.2002	**TEM** ELECTRIC MOTORS s.r.l		M.06.05 REV.0
DATI MOTORE Motor ratings	SIMBOLI Symbols	UNITA' Units	**PMB 1524**
COPPIA ALLA VELOCITA' NOMINALE Torque at rated speed	Cn	Nm	0.22
VELOCITA' NOMINALE Rated speed	Nn	RPM	1500
POTENZA NOMINALE Rated output	Pu	W	35
TENSIONE NOMINALE Rated voltage	Vn	V	24
CORRENTE NOMINALE Rated current	In	A	1.9
VELOCITA' A VUOTO Idle speed	Nv	RPM	1750
TENSIONE A VUOTO Idle voltage	Vv	V	24
CORRENTE A VUOTO Idle current	Iv	A	0.3
F.C.E.M ALLA VELOCITA' NOMINALE B.E.M.F at rated speed	E	V	19.5
COPPIA DI PICCO Peak torque	Cp	Nm	0.88
CORRENTE DI PICCO Peak current	Ip	A	8
MAX VELOCITA' Max speed	N max	RPM	4000
RENDIMENTO Efficiency	η	%	73
DATI MECCANICI Mechanical data			
MOMENTO D'INERZIA ROTORICO Rotor inertia	J	Kgcm²	0.352
MAX. ACCELLERAZ. TEORICA Max theorical accelleration	α	rad/sec²	25000
MAX TENSIONE Max voltage	V max	V	65
COSTANTE DI TEMPO MECCANICA Mechanical time constant	Tm	ms	85
VENTILAZIONE Ventilation			Naturale
GRADO DI PROTEZIONE Protection (IEC 34.5)		IP	54
PESO Weight	G	Kg	1.35
DATI ELETTRICI Electric data			
COSTANTE DI TENSIONE Voltage constant	Ke	Vs\rad	0.12
COSTANTE DI COPPIA Torque constant	Kt	Nm\A	0.11
COSTANTE DI TEMPO TERMICA Thermal time constant	Tt	min	15
COSTANTE DI TEMPO ELETT. Electrical time constant	Te	ms	1.3
RESISTENZA D'ARMATURA Armature resistance	Ra	Ohm	--
RESISTENZA D'ARMATURA CON SPAZZOLE Terminal resistance	Rm	Ohm	3.20
INDUTTANZA D'ARMATURA Armature inductance	La	mH	4.1
TEMPERATURA AVVOLGIMENTI Winding temperature	Ta	°C	--
TEMPERATURA COLLETTORE Commutator temperature	Tc	°C	--
CLASSE ISOLAMENTO Insulation class			F
FATTORE DI SERVIZIO Duty			S1
FATTORE DI FORMA Form factor			1
TEMPERATURA AMBIENTE Ambient temperature		°C	25
ALTEZZA Height		m	1000
TOLLERANZE Tollerance		%	±5
DATI AVVOLGIMENTO Winding data			
DIAMETRO ESTERNO LAMIERINO External diameter laminator	Ø	mm	35.5
ALTEZZA PACCO Package height	H	mm	52 + T
N° CAVE N°slots		N°	11
FORO ASSE Hole axis	Ø	mm	9
PASSO AVVOLGIMENTO Winding step			1 - 6
SPIRE CAVA Slots coil		N°	80
SPIRE USCITA Exit coil		N°	40
USCITE CAVA Exit slots		N°	1
FILO Ø Wire Ø	Ø	mm	0.375 D
TIPO COLLETTORE Commutator type			11-21-11-9
PROFONDITA' COLLETTORE Commutator depth		mm	4.5 / 5 mm
SPOSTAMENTO COLLETTORE Commutator displacement			

Figure 3.2: Motor ratings

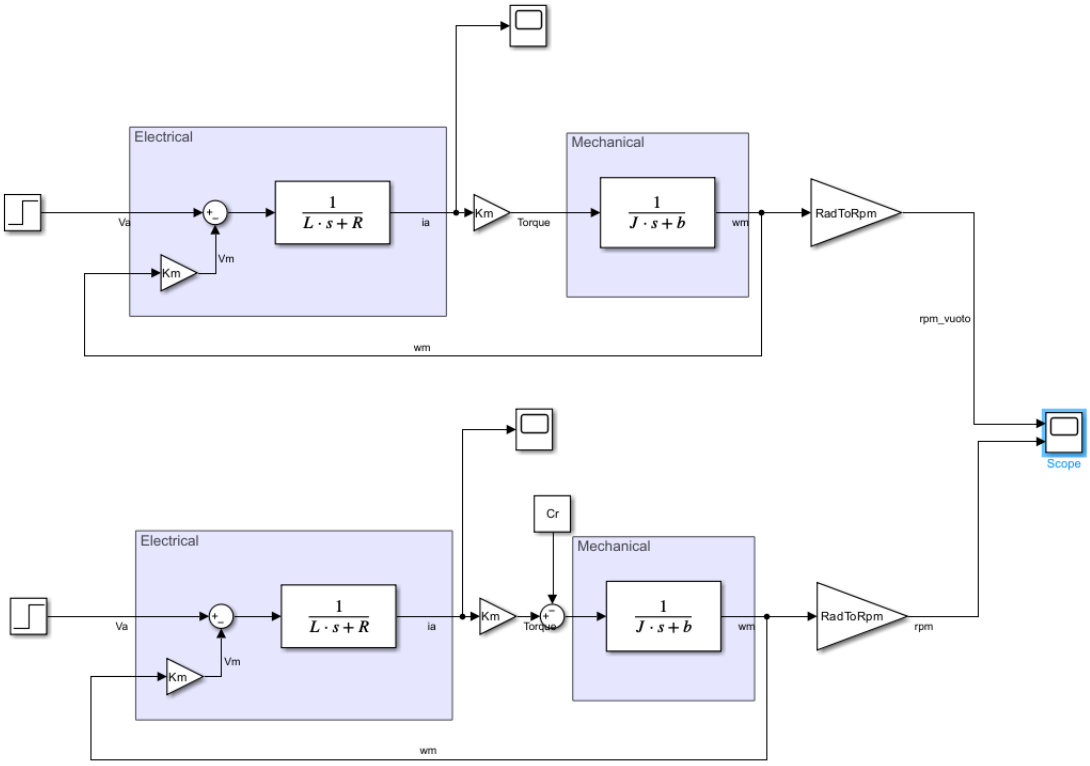

Figure 3.3: 2 motor block scheme in simulink: upper one is the scheme without considering a resistive torque (breaking resistance), the one below instead, include the braking resistence

Chapter 4

Simulink Schemes and Results

The Simulink implementation of the motor model was excited using the nominal voltage specified in the datasheet, i.e. 24 V. The model reproduces both the *no-load* and *rated-load* operating conditions to verify consistency with the manufacturer's data.

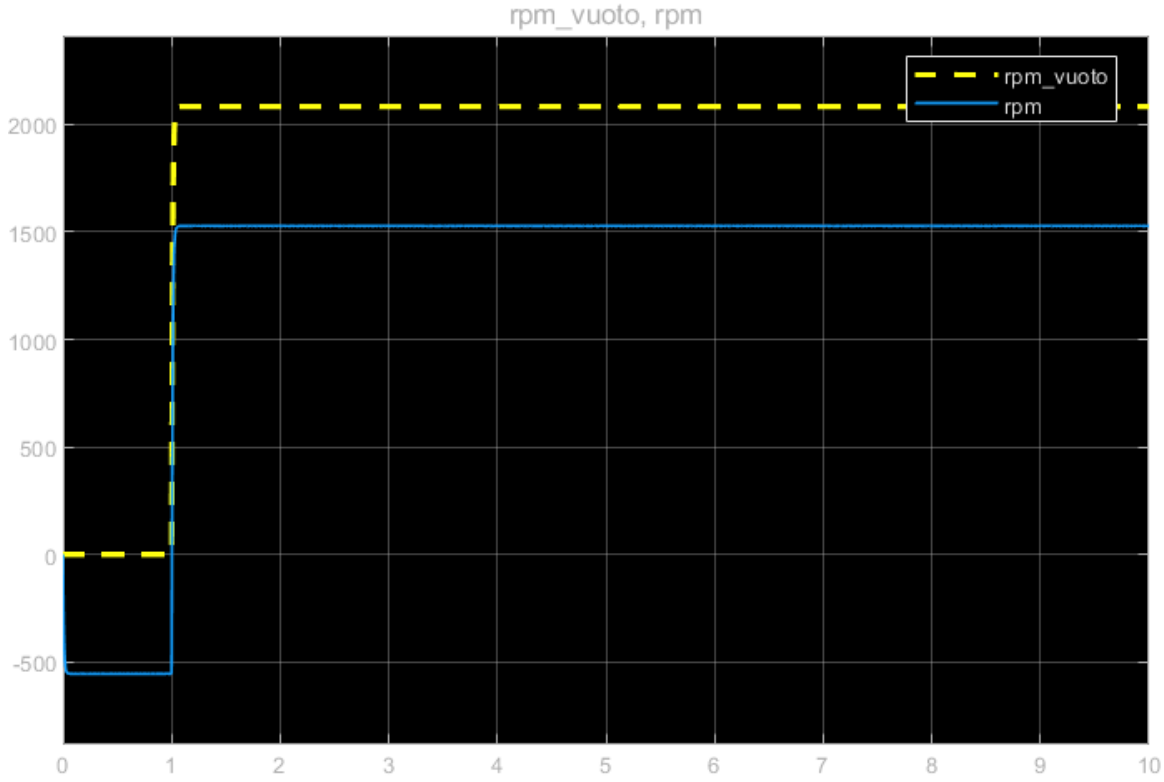

Figure 4.1: Simulated motor velocities under no-load (yellow) and rated-load (blue) conditions.

4.1 Velocity Response Analysis

As shown in Fig. 4.1, the blue curve represents the rated-speed condition, achieved by applying a resistive torque equivalent to the rated torque value listed in the datasheet. The simulated steady-state speed of 1520 rpm closely matches the nominal 1500 rpm, with the minor discrepancy attributed to numerical approximations and the simplified loss model that neglects friction and windage effects.

The yellow curve corresponds to the no-load operating condition. Here, the predicted speed exceeds 2000 rpm, compared to the datasheet reference of 1750 rpm. This deviation occurs because the friction coefficient b was set to zero in this preliminary simulation, eliminating resistive torque effects. Additionally, the plot reveals a brief negative speed region at $t < 1$ s, caused by the absence of electromagnetic torque before voltage application. During this transient, the residual resistive torque momentarily drives the shaft in the opposite direction.

4.2 Current Response Analysis

The electrical behavior is shown in Fig. 4.2, where both the nominal and no-load currents are compared.

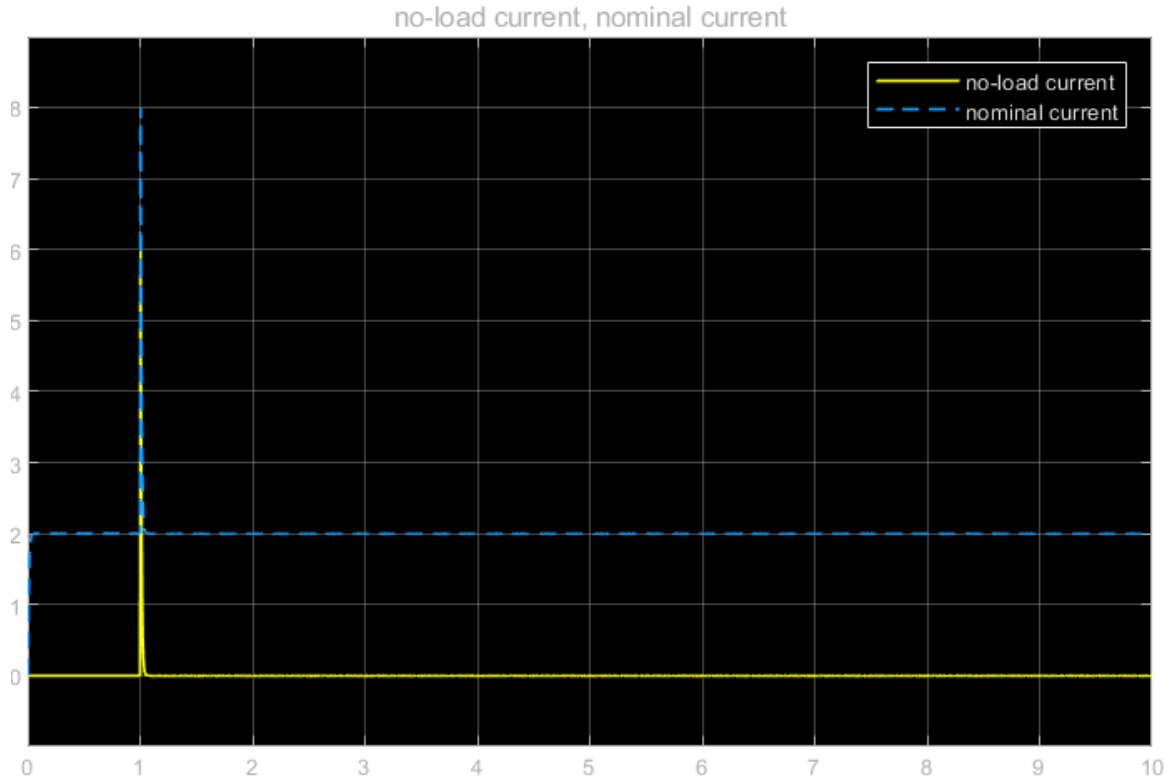

Figure 4.2: Motor current responses under nominal and no-load conditions.

The steady-state no-load current is nearly zero, as it primarily compensates for

mechanical friction losses—which were neglected in this model. In real systems, a small residual current (around 0.3 A) is observed, consistent with the datasheet. Under nominal operation, the current stabilizes around 2 A, slightly above the rated 1.9 A, due to the applied resistive torque.

A transient current spike, peaking near 8 A, appears at the voltage step transition. This behavior is consistent with the motor's inductive nature, where the winding opposes sudden changes in current flow. The magnitude of this spike agrees with the datasheet specification, validating the electrical model.

To generalize the analysis, the Simulink model was reformulated in terms of transfer functions. This abstraction allows investigation of the system's dynamic behavior without direct reference to physical parameters.

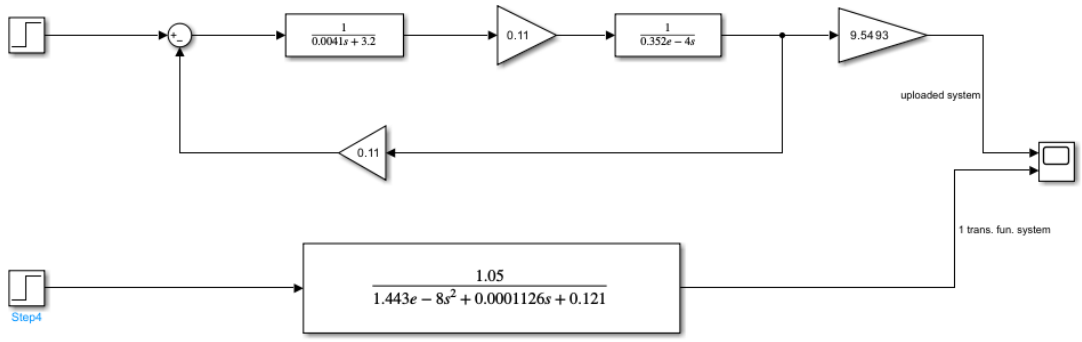

Figure 4.3: Simulink implementation using numerical transfer functions.

4.3 Dynamic Comparison and Discussion

The responses of the two equivalent systems (physical model vs. transfer-function model) are compared in Fig. 4.4. Their behavior is nearly identical, demonstrating the validity of the numerical representation. Both responses exhibit a first-order dominant dynamic due to the small coefficient multiplying the second-order term.

A slight discrepancy can be observed in the initial transient phase, as highlighted in Fig. 4.5. This minor deviation originates from numerical precision and the absence of unmodeled nonlinear effects.

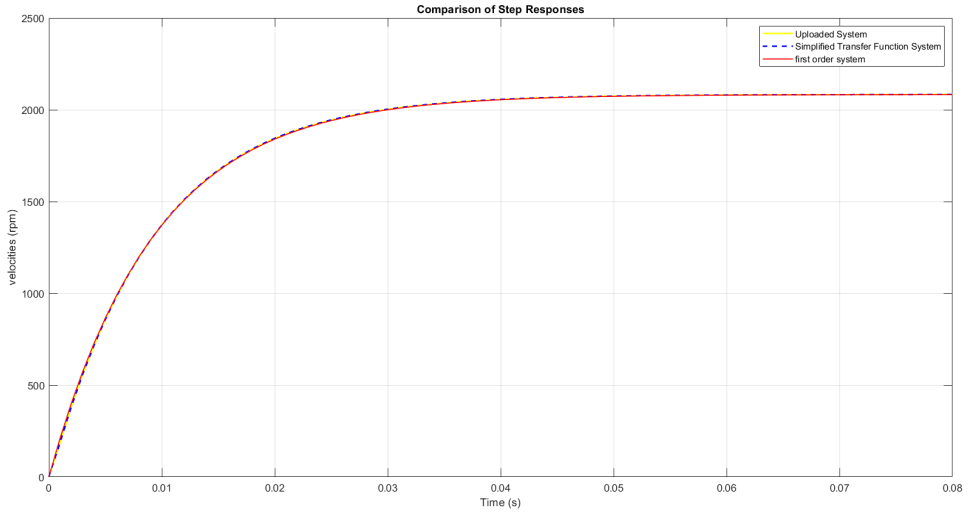

Figure 4.4: Comparison between physical and transfer-function-based velocity responses.

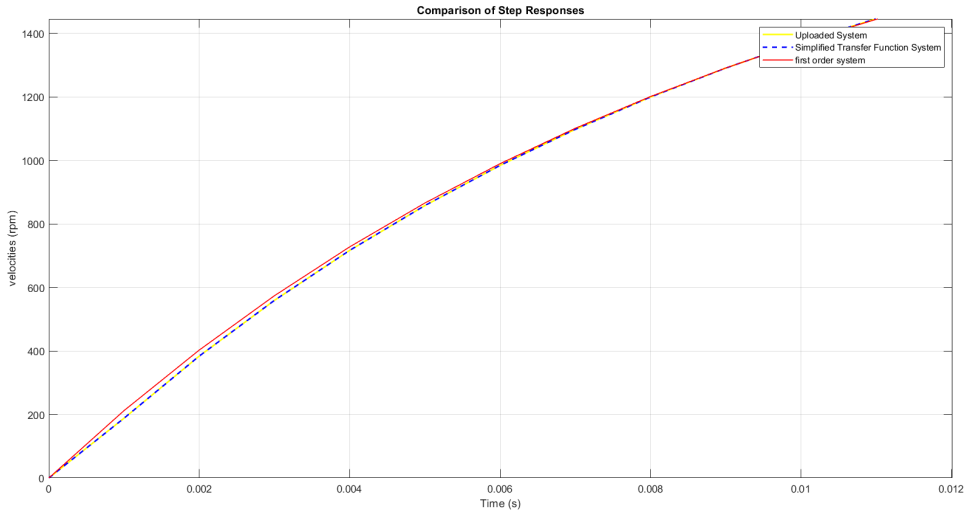

Figure 4.5: Initial transient comparison between the two simulation models.

Chapter 5

Discretization and Encoder Processing

The model designed for the DC motor must be discretized to ensure compatibility with the microprocessor on the control board. Since the microprocessor operates in the digital domain, it processes discrete-time signals, making discretization an essential step before hardware implementation.

5.1 Discretization of the Motor Model

Discretization converts continuous-time differential equations into difference equations that can be processed numerically. Among the most common techniques are the *Zero-Order Hold* (ZOH) and the *Tustin* (bilinear transformation) methods. In this study, only selected state variables were discretized and exported to the MATLAB workspace through the `To Workspace` block for further analysis.

In Simulink, the `Zero-Order Hold` block was used to discretize the chosen signals, ensuring accurate representation of the continuous-time dynamics in a discrete-time framework. The resulting model, adapted for digital computation, is shown in Fig. 5.1.

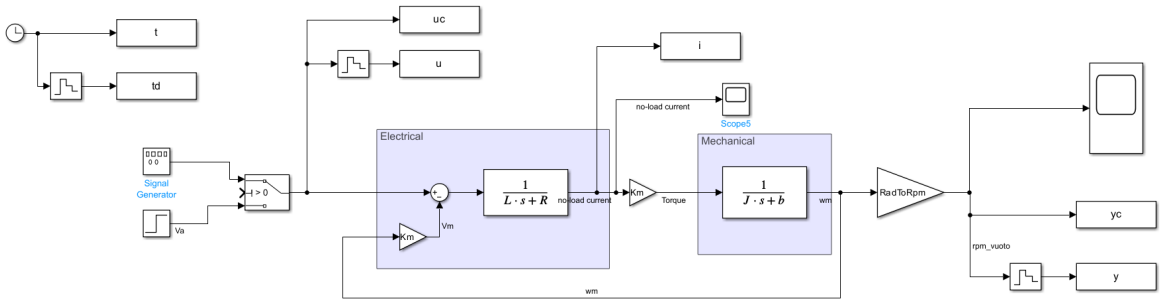

Figure 5.1: Modified Simulink model including discrete-time variable acquisition.

5.2 Time Constant Estimation

The variables retrieved from the discrete-time simulations were analyzed to estimate the system's time constant and to compare the actual response with its first-order approximation (Fig. 5.2).

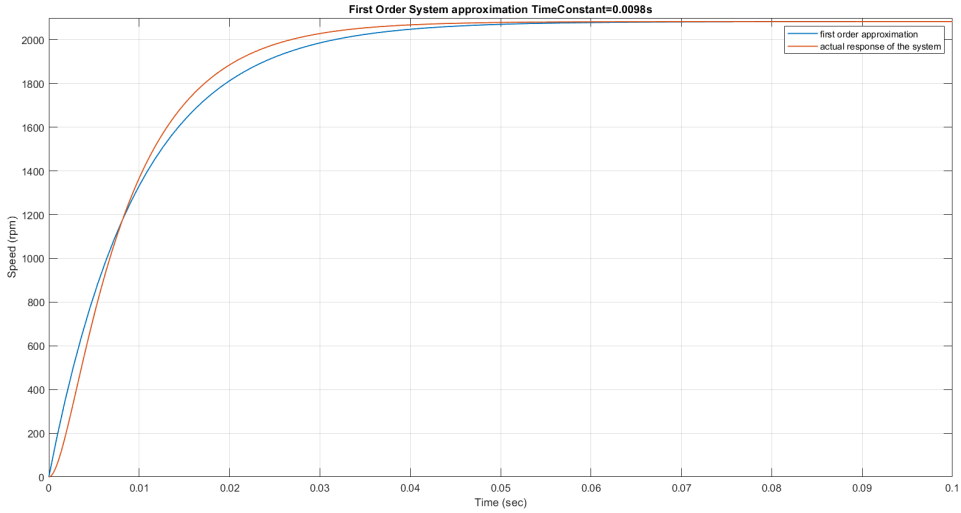

Figure 5.2: Comparison between the actual system response and its first-order approximation.

The time constant τ was derived following a standard procedure:

- The maximum steady-state value of the response was identified from the exported data.
- This value was multiplied by 0.6321, since in first-order systems the time constant is the time required for the response to reach 63.2% of its final value after a step input.
- The corresponding time instant was located in the time vector to determine τ.

5.3 Encoder Signal Processing

The experimental setup includes a quadrature encoder, a sensor that converts the rotational motion of the motor shaft into electrical signals. The encoder used has a resolution of 2048 counts per turn, equivalent to 8192 counts per full revolution when considering both channels. The motor speed is computed by evaluating the change in encoder counts (ΔCNT) over a defined sampling period (Δt).

5.3.1 Motivation for Filtering

Before performing experiments on the physical system, simulations were conducted to highlight the importance of filtering the encoder output. Noise and quantization effects can significantly affect speed estimation, particularly when differentiating the position signal. Filtering improves signal reliability and ensures synchronization between the encoder's sampling rate and the system's PWM frequency.

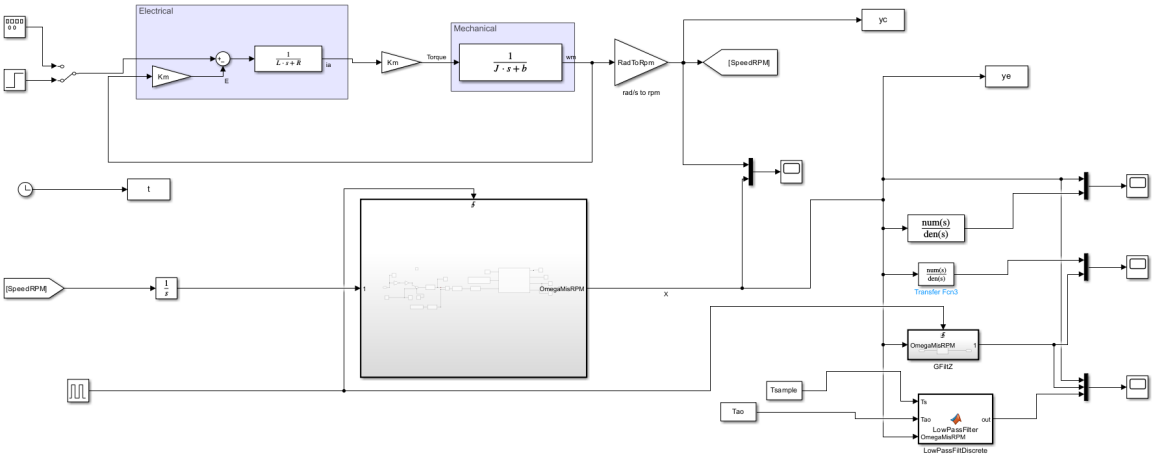

Figure 5.3: Simulink scheme for encoder signal processing and PWM synchronization.

5.4 Filtering of the Encoder Signal

In real applications, the velocity signal derived from the encoder is affected by multiple noise sources:

- Mechanical vibrations and shaft oscillations;
- Electrical interference from the motor drive;
- Quantization errors introduced during digital sampling.

These disturbances contaminate the raw signal—represented in Simulink by the subsystem labeled `OmegaMisRPM`—and must be mitigated through appropriate filtering.

5.4.1 Implemented Filtering Techniques

Three filtering methods were implemented and compared:

- A continuous first-order low-pass filter with transfer function:

$$G(s) = \frac{1}{\tau s + 1},$$

  where $\tau = 0.01$ and the sampling period is $T_s = 0.0001\,\text{s}$.

- The same filter discretized via the Tustin (bilinear) method in MATLAB, with the resulting discrete numerator and denominator coefficients imported into Simulink as a `Transfer Fcn` block.
- A discrete filter obtained through the backward-difference method, implemented directly in Simulink using a `MATLAB Function` block.

5.4.2 Comparison of Results

The filtering methods were evaluated in simulation, and their performance is shown in Fig. 5.4. All approaches effectively reduce noise and enhance the reliability of the speed signal, confirming the necessity of filtering in encoder-based velocity feedback.

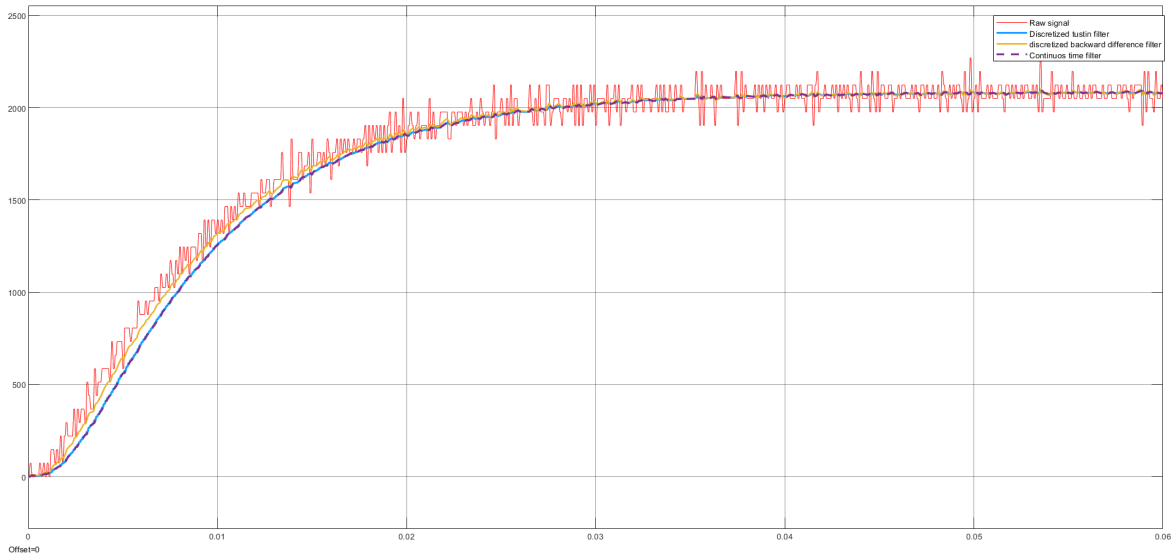

Figure 5.4: Comparison of filtered encoder outputs using different methods.

Filtering is therefore indispensable for achieving accurate and stable control. In the experimental system, one of these methods must be applied to the encoder output to ensure that the measured velocity remains consistent with the real mechanical behavior of the motor.

Chapter 6

Interface and Cascade PI Control

A preliminary real-time interface is introduced to control the motor and monitor key variables such as current (I), voltage (V), and angular velocity (Ω). The interface is managed by a *C2000 Microcontroller* block—specifically the `C28x Interrupt` block—which creates an interrupt service routine (ISR) to execute the downstream subsystem. This interrupt-based architecture ensures responsive control and precise execution of motor tasks.

6.1 Subsystems in the Real-Time Interface

6.1.1 Analog Reading

Raw analog signals are acquired via the C2000 blocks and conditioned as follows. The motor phase current signal `IsenA` is offset-compensated using `OffsetADCINC4` to remove measurement DC bias. After offset removal, a Low-Pass Filter attenuates high-frequency noise. Filtered ADC counts are then converted to voltage using the ADC reference and resolution, and finally to current using the shunt resistance and sensor gain. The DC-link voltage `Vdc` is similarly converted from ADC counts to voltage using the sensor gain. Motor phase voltages (`VsenA`, `VsenB`) are processed and combined to obtain the total motor voltage `Vmot`.

Digital inputs made available to the control layer are:

- Motor current (`IsenA`),
- Motor voltage (`Vmot`),
- DC-link voltage (`Vdc`).

6.1.2 Encoder Section

The previously demonstrated encoder processing (filtering and rate alignment) is adapted here to operate on the *actual* signal from the control board rather than a

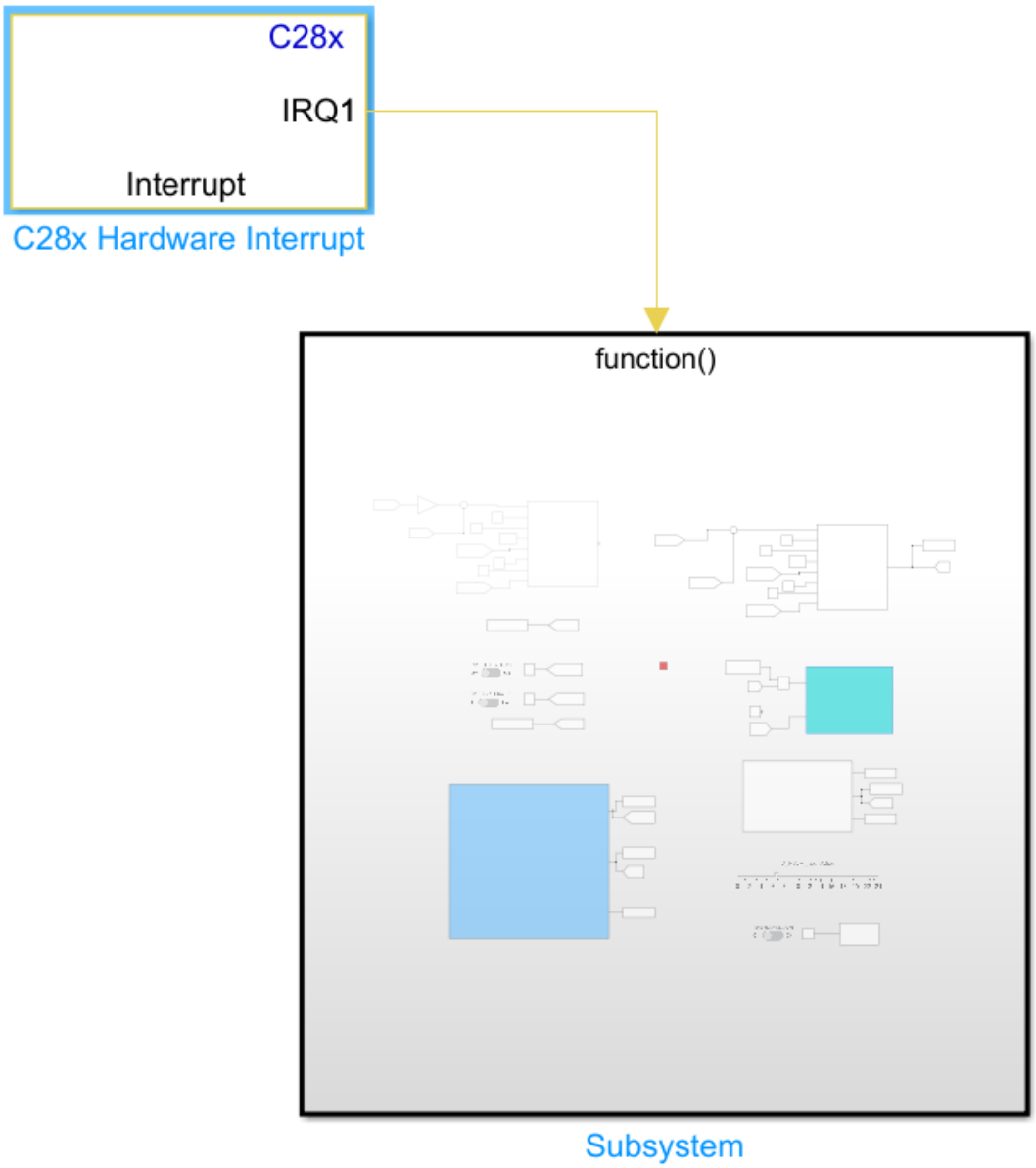

Figure 6.1: Preliminary control and monitoring interface managed by the C28x ISR.

simulated input, ensuring a usable velocity estimate for feedback.

6.1.3 PWM Signal Generation

PWM signals are generated from the requested voltage (controller output) and the power supply voltage (nominally 24 V). The requested voltage is normalized to the duty-cycle range $[-1, +1]$ via a saturation block; the result is scaled/offset to produce the duty cycle. Two PWM outputs are used for the H-bridge: a direct signal (`PWM MOT A`) and its complementary (`PWM MOT B`); the third output (`PWM MOT C`) is set to zero.

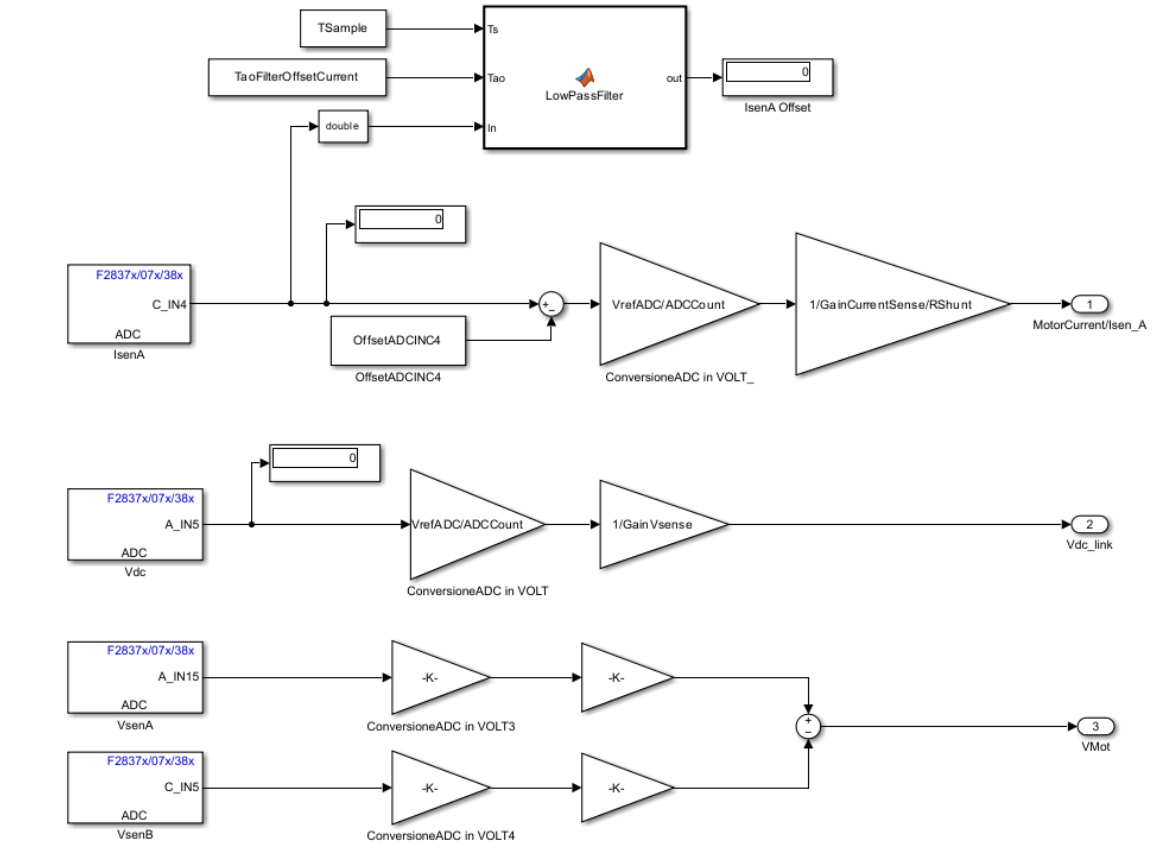

Figure 6.2: Analog reading and conditioning section.

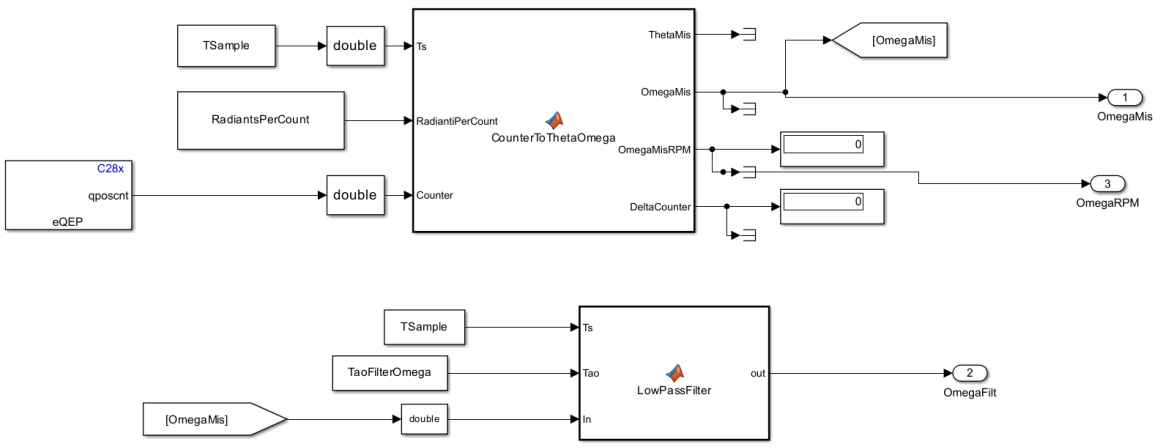

Figure 6.3: Encoder signal processing section.

6.2 Cascade Control Architecture

Speed control is realized with a cascade architecture: the inner loop regulates current (torque), and the outer loop regulates velocity. Neglecting inner-loop dynamics is justified by *dynamic separation*: the inner loop must be significantly faster than the outer loop (higher bandwidth). Here, the inner-loop electrical time constant is

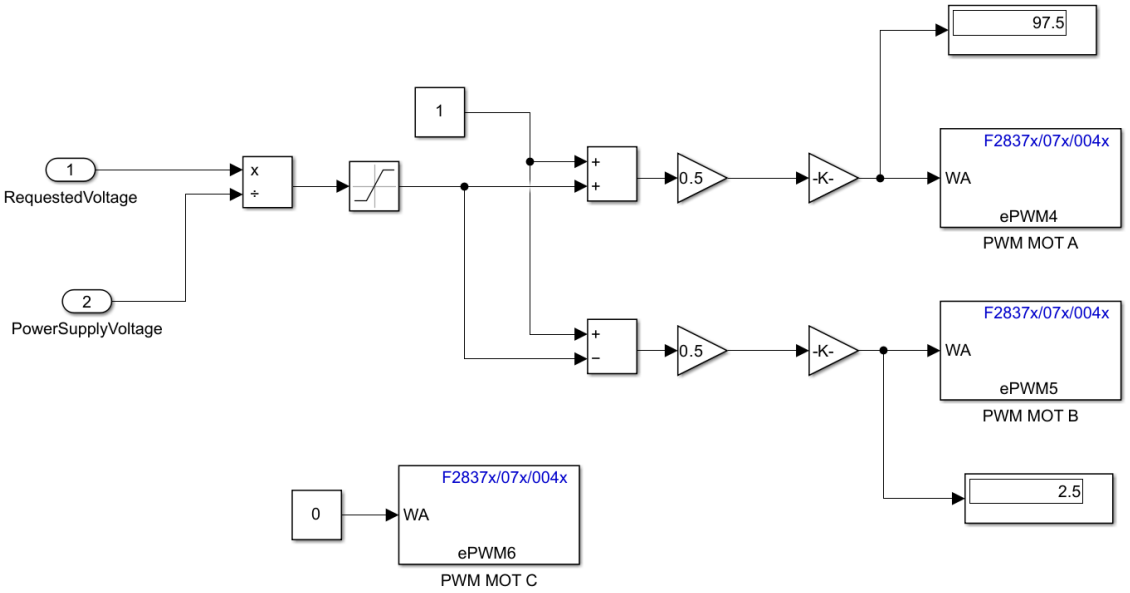

Figure 6.4: PWM signal generation for the motor driver.

1.3 ms (datasheet), while the outer-loop mechanical time constant is 85 ms, ensuring effective separation.

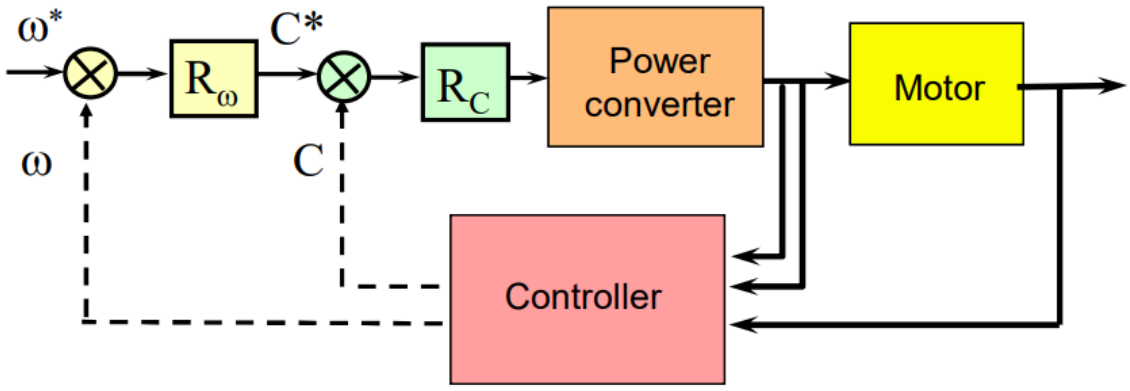

Figure 6.5: Cascade controller architecture (current inner loop, velocity outer loop).

6.2.1 Current Loop with Back-EMF Compensation

The current loop controls torque via the static relation $\tau(t) = k_m i(t)$. The goals are zero steady-state error (integral action) and fast response (proportional action), hence a PI controller:

$$R_i(s) = k_{pi}\left(1 + \frac{1}{T_{ii}s}\right),$$

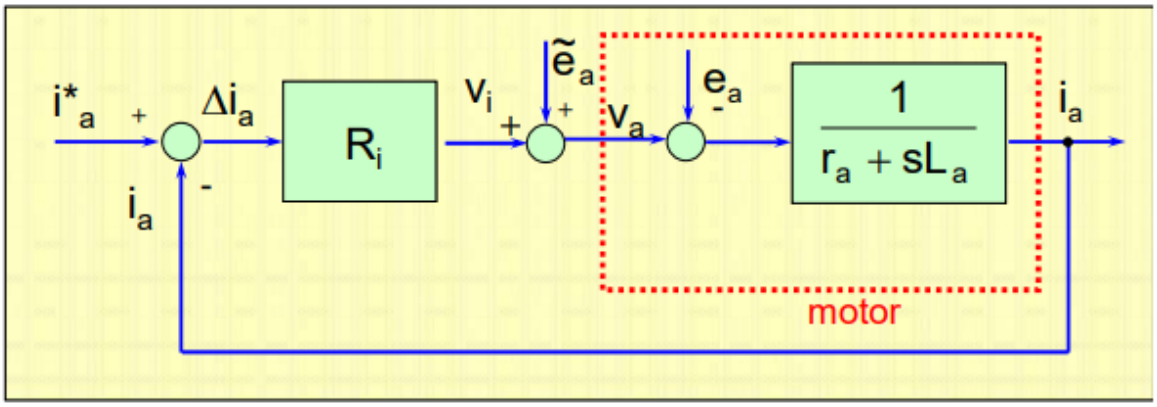

Figure 6.6: Current regulator with back-EMF compensation.

with proportional gain K_{pi} and integral time constant T_{ii}. Combined with the motor's electrical dynamics:

$$G_i(s) = R_i(s)G_e(s) = \frac{K_{pi}(1 + T_{ii}s)}{T_{ii}s} \cdot \frac{1/R_a}{1 + sT_a}.$$

Set $T_{ii} = \tau_a$ (electrical time constant) to cancel the motor electrical pole. Choose

$$K_{pi} = \frac{L_a}{\tau_i},$$

so that the closed-loop function becomes

$$F_i(s) = \frac{1}{1 + \frac{R_a T_{ii}}{K_{pi}} s} = \frac{1}{1 + \tau_i s}.$$

Thus $K_{pi} = L_a/\tau_i$ tunes the response speed; τ_i too small yields very fast but potentially poor behavior, so a trade-off is required. Values near the electrical time constant are typically best.

6.2.2 Velocity Loop PI Design

With the inner loop tuned, it can be represented as

$$F_i(s) = \frac{1}{1 + \tau_i s}.$$

The velocity controller is then designed per the block diagram in Fig. 6.7.

The factor $\frac{1}{k_m}$ ensures that the output of $R_\omega(s)$ can be interpreted as the torque applied to the mechanical subsystem $\frac{1}{b+Js}$; this term should be included in $R_m(s)$. A PI regulator is implemented:

$$R_\omega(s) = \frac{K_{p\omega}}{T_{i\omega}} \left(\frac{1 + T_{i\omega}s}{s} \right),$$

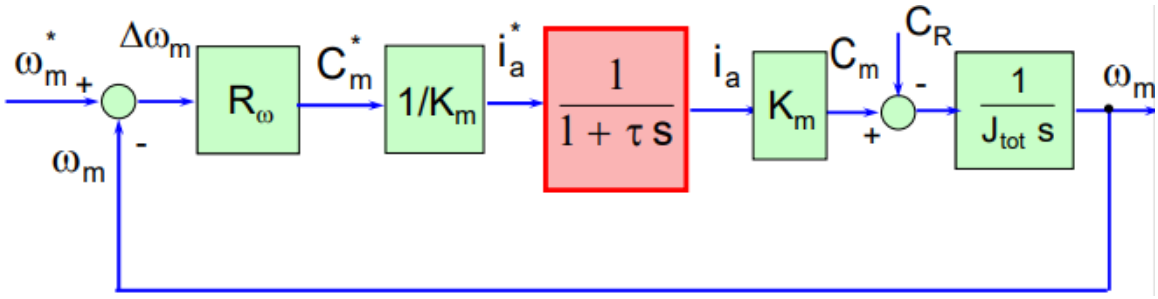

Figure 6.7: Block diagram of the velocity control loop.

with $T_{i\omega} = \tau_m = \frac{J}{b}$, canceling a plant pole. Neglecting the much faster inner loop yields the closed-loop function

$$F_\omega(s) = \frac{1}{1 + \frac{bT_{i\omega}s}{K_{p\omega}K_m}} = \frac{1}{1 + \tau_\omega s},$$

and therefore

$$K_{p\omega} = \frac{bT_{i\omega}}{\tau_\omega K_m} = \frac{J}{\tau_\omega K_m}.$$

To preserve dynamic separation, τ_ω must exceed τ_i; here, $\tau_\omega = 9\,\tau_i$.

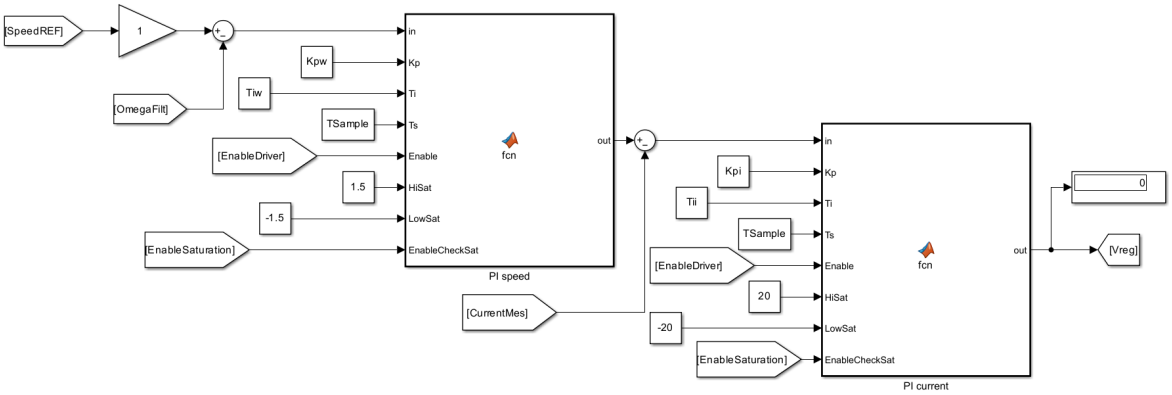

Figure 6.8: Simulink schematic of the cascade PI controller.

PI current and velocity controllers are discretized inside `MATLAB Function` blocks using the Tustin method, ensuring consistency with the digital execution context.

To assess behavior under varying inner-loop speeds, multiple tests were run with different τ_i values. All tests shared the same conditions:

- Nominal supply voltage: 24 V,
- No load attached (braking resistor disconnected),
- Step reference with steady value 1000 rpm,
- All controller parameters fixed except τ_i.

6.3 Results: Effect of τ_i on Closed-Loop Behavior

6.3.1 Very Fast Inner Loop: $\tau_i = 0.0001\,\text{s}$

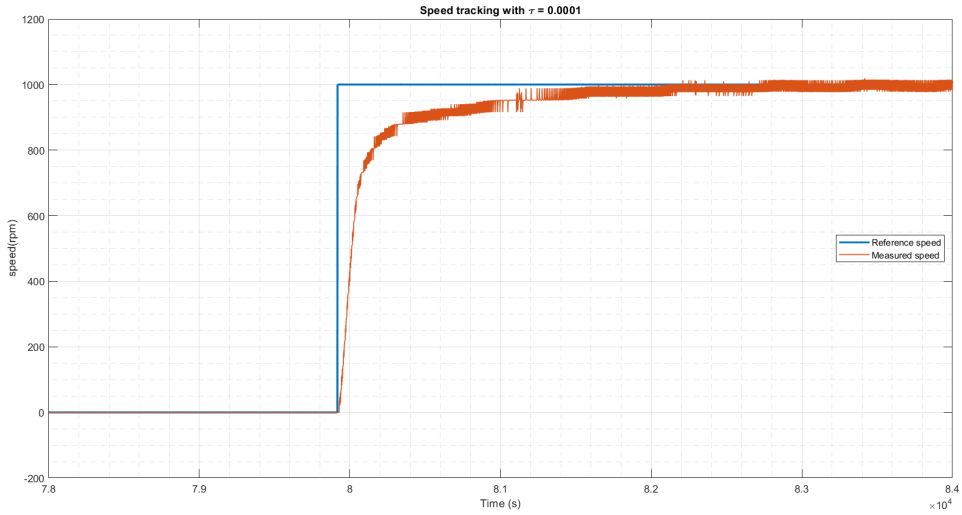

Figure 6.9: Step response with $\tau_i = 0.0001\,\text{s}$.

The response resembles a first-order system (inner-loop dynamics effectively negligible), with no oscillations. However, significant measurement noise appears. The system reaches steady state in $0.3 \times 10^{-4}\,\text{s}$, i.e., more than ten times faster than the electrical time constant. The mean value tracks the reference with zero steady-state error, as expected from integral action.

6.3.2 Slow Inner Loop: $\tau_i = 0.01\,\text{s}$

Noise is absent, but oscillations of about 8% of steady-state appear. The settling time is comparable to the previous case.

6.3.3 Time Constant Near Electrical Dynamics: $\tau_i = 0.001\,\text{s}$

Choosing τ_i close to the electrical time constant (1.3 ms) yields a small overshoot ($\approx$ 2%) and minimal noise, offering a favorable trade-off between speed and robustness.

With the step-response analysis complete, the system can now be evaluated against more complex reference trajectories to assess tracking under time-varying conditions.

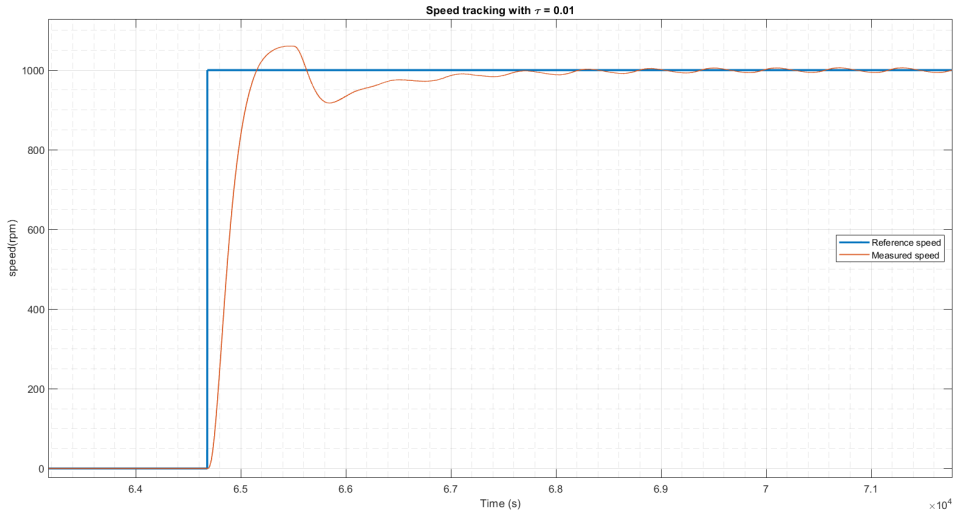

Figure 6.10: Step response with $\tau_i = 0.01$ s.

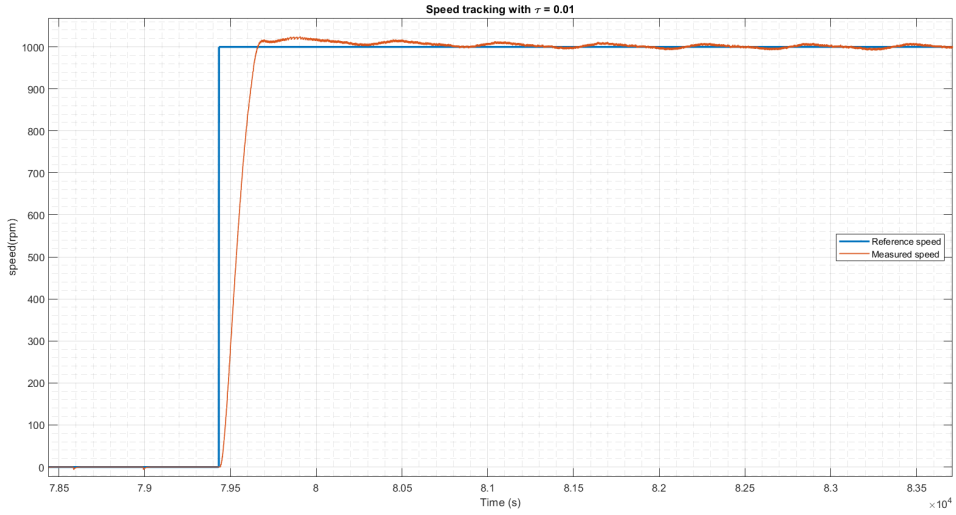

Figure 6.11: Step response with $\tau_i = 0.001$ s.

Chapter 7

Feedforward Action and Anti-Windup

Before proceeding, a second interface was introduced because the existing setup only allowed observation of the steady-state values and not the transients, making it difficult to analyze the dynamic behavior of the control system.

7.1 Upgraded Framework: Control and Communication

In the upgraded framework, the control section is implemented in a Simulink file, while the communication section is handled in a separate file. The control part consists of the same blocks previously described, with some additional components (Fig. 7.1). Notably, a debug system is included within the control implementation subsystem (Fig. 7.2). This unit is essential for sending variables back to the communication board for observation and further analysis.

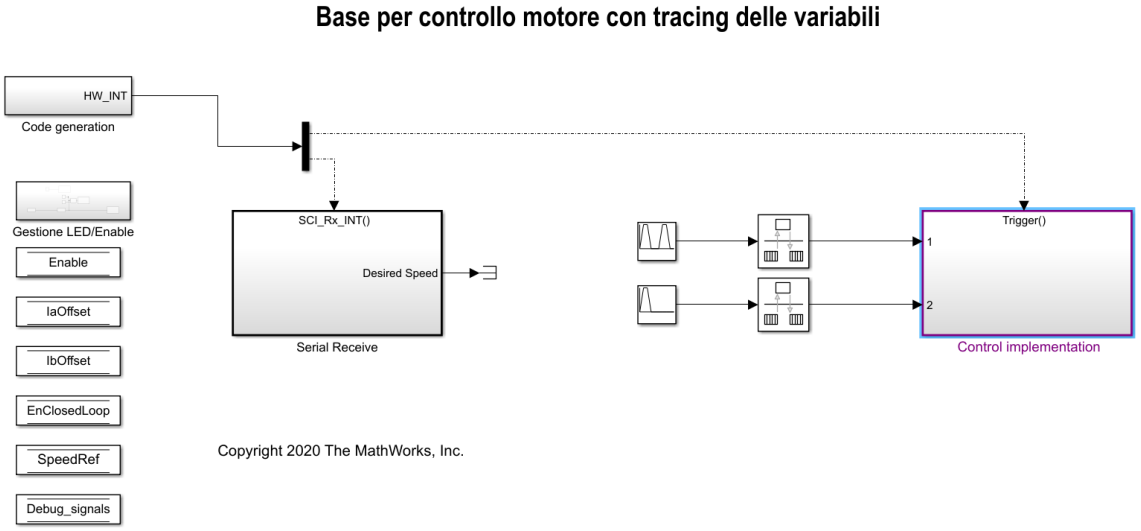

Figure 7.1: Control interface with added modules for data export and observability.

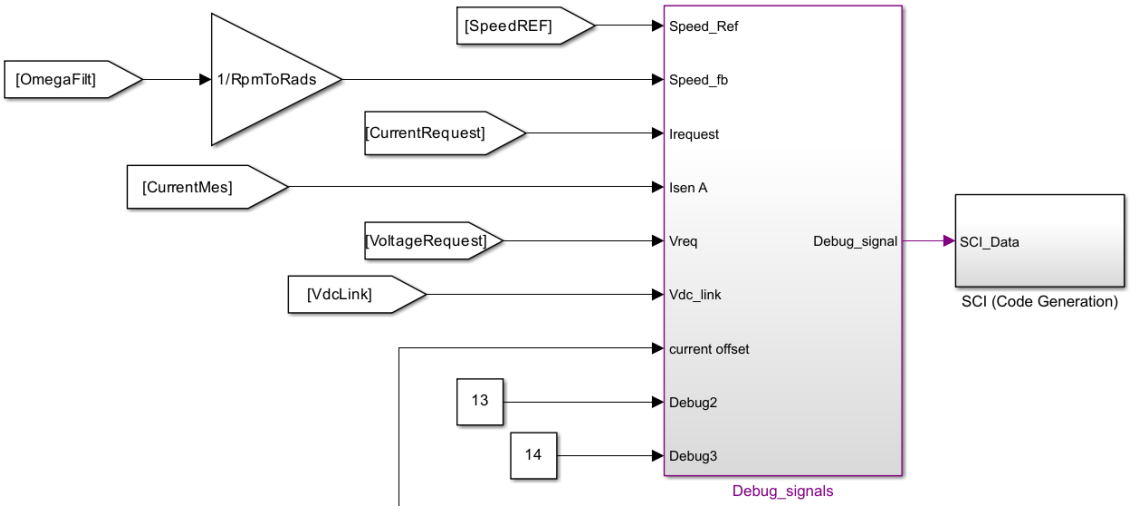

Figure 7.2: Debug system unit for logging and streaming selected variables.

Communication Dashboard

The communication dashboard (Fig. 7.3) is totally new and contains the following elements:

- **Reference speed (RPM):** a block available to set a constant reference speed that can eventually be tracked by the control dashboard;
- **Tx subsystem:** a module needed to transmit the operation of this interface to the control one;
- **ON/OFF Button:** a switch needed to start and stop the motor;
- **Rx subsystem:** a module needed to receive the data from the control dashboard;
- **Debug signals:** a unit that makes possible to choose the 2 signals to analyze and examine.

7.2 Tracking Under Sinusoidal References

Different kinds of reference trajectories could be given to evaluate the behavior of the system under variable situations and conditions. In Fig. 7.4 is plotted the response to a Sin Wave reference trajectory with a frequency of 25 kHz without the connection of the braking load.

The tracking performance, while not perfect, is acceptable given the high frequency of the reference signal. As anticipated from the step response, the actual response exhibits a minimal overshoot. The plot demonstrates that the maximum amplitude (both positive and negative) is slightly higher than the desired reference.

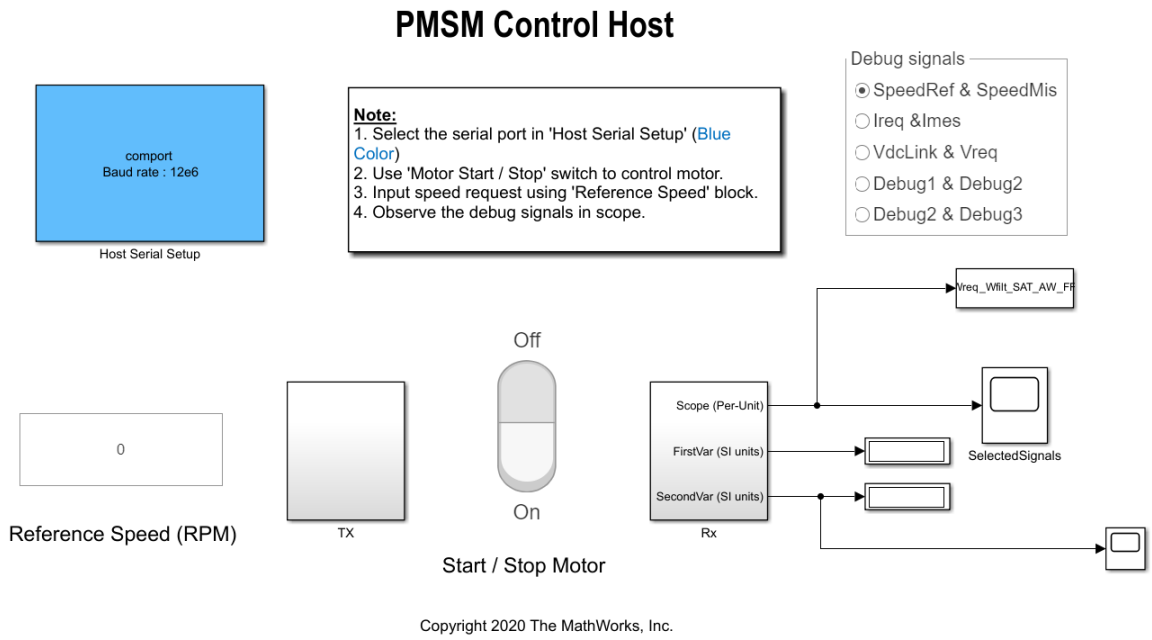

Figure 7.3: Communication interface for reference injection, control, and monitoring.

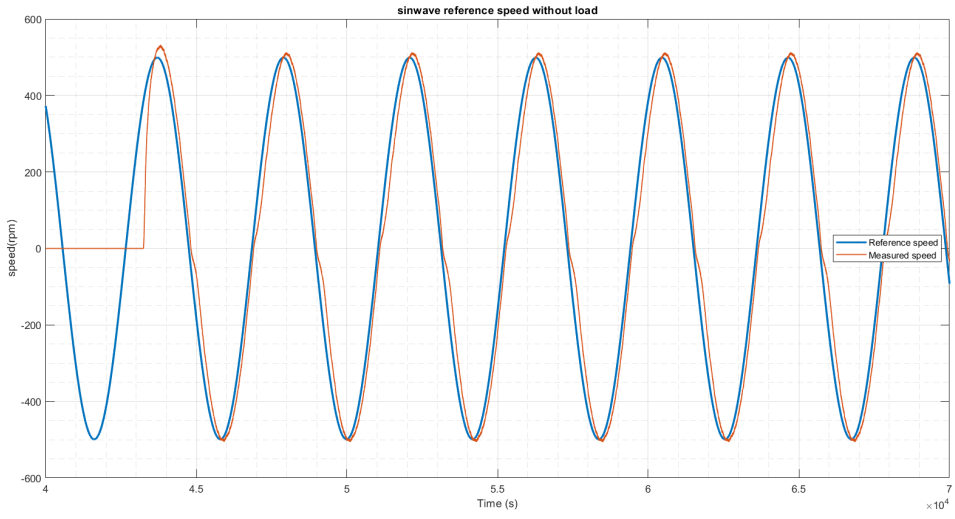

Figure 7.4: Sinusoidal reference trajectory without load.

Another effect of the high pulsation of the sinusoidal wave is the lack of a null error at any point in the graph. This occurs because the system's dynamics are not fast enough to fully track the variations of the reference. Furthermore, it can be observed that the error reaches its maximum value during the change of direction. This behavior is expected since any physical system subject to rotation must dissipate the inertia energy from motion in one direction before it can rotate in the opposite direction. This highlights the limitations of the motor's dynamic response under such conditions.

In Fig. 7.5, the response to the same reference trajectory is shown, with the only difference being the connection of a load (braking resistance).

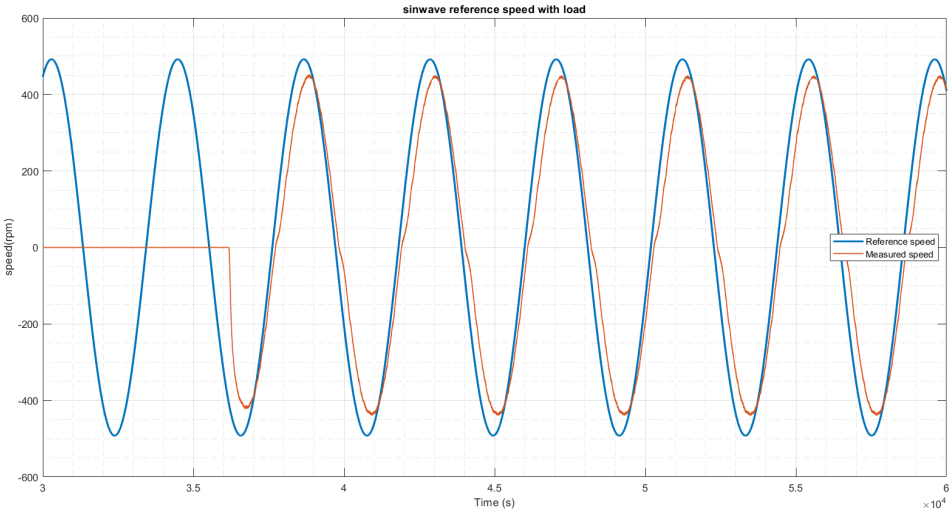

Figure 7.5: Sinusoidal reference trajectory with load.

In this case, the system fails to reach the maximum (and minimum) values of the reference signal because the controller is tuned for the base motor model. This tuning leads to a slower response and an error, which also accounts for the increased inertia of the system.

From the system run, several variables were saved and analyzed, including the requested voltage from the control system and the actual voltage measured from the setup. A comparison between the no-load and with-load conditions is shown in Fig. 7.6. As expected, the measured voltage for the motor alone is smaller than the voltage for the motor with the braking resistor configuration.

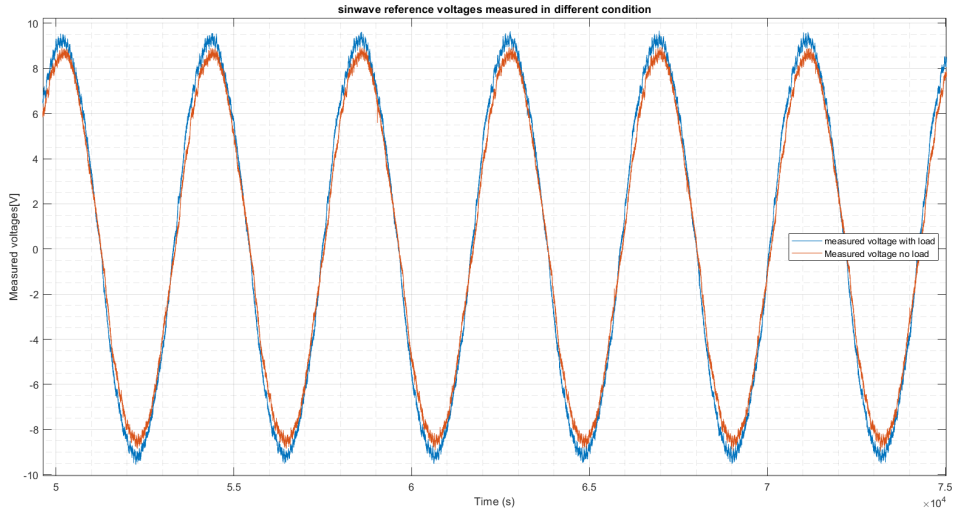

Figure 7.6: Comparison of measured voltages: no-load vs. load.

7.3 Feedforward Action: Rationale and Integration

To address increasing performance demands and prevent degradation in tracking capabilities, a feedforward action should be introduced. The main drawback of this architecture is that it requires precise knowledge of the plant as well as noise signals, which means it must be integrated with feedback control and cannot function as a stand-alone solution.

Nevertheless, the combined design offers several advantages:

- Proper filtering of the reference input by modifying the transfer function between the desired input y_d and the output y, to achieve predefined static or dynamic properties for the system;
- Improved tracking performance;
- Compensation for known disturbances.

The schematic of this architecture is shown in Fig. 7.7, highlighting the presence of a pre-filtering action, denoted as $C_f(s)$.

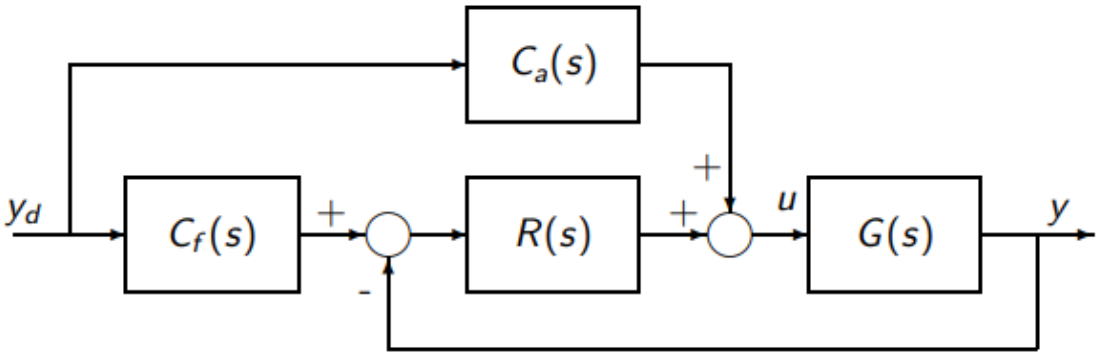

Figure 7.7: Feedforward action integrated in feedback control with pre-filter $C_f(s)$.

Simulink Realization in the Cascade Framework

In our case, given that the control implemented is structured by the way of a cascade design, the feedforward term could interact with both loops (current and velocity); this construct can be observed in Fig. 7.8.

The input to the feedforward action, represented by input number "2," is the `SpeedREF` variable, which accounts for the reference speed trajectory. The current requested by the speed PI control block is integrated with the current provided by the feedforward action. This integration reduces the significant error in the initial phase of feedback evaluations, thereby enhancing the system's response.

The speed feedforward action can also be interpreted as an estimation of the back electromotive force (EMF), which constitutes the dominant term of the armature

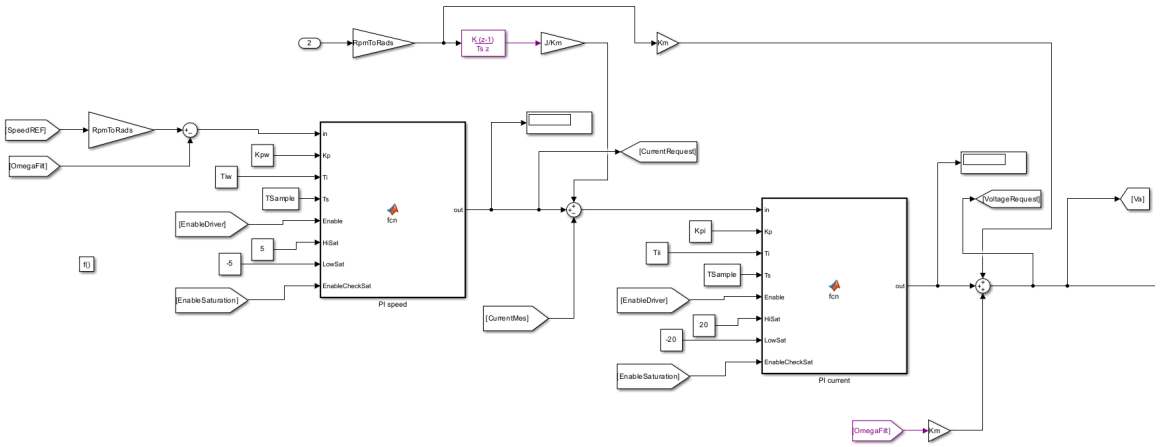

Figure 7.8: Simulink scheme of feedforward integrated action in the cascade loops.

voltage except at very low speeds. Accurate back EMF estimation is critical, as it directly influences the control accuracy and overall performance.

7.4 Actuator Limits and Anti-Windup

Another essential consideration is the actuator saturation effect, which can occur when the control action is overly aggressive. If neglected, this can lead to closed-loop instability, as the control action may remain constant for extended periods, despite varying output values computed by the controller.

In this particular scenario, the actuator has limits on the maximum and minimum power supply values it can access. These limits are set on the generator at 30 volts for the supply voltage and 6 amperes for the supplied current. When combined with the integral action of the feedback control, these constraints can result in the wind-up phenomenon. To prevent undesired effects, such as overshoot, prolonged recovery times, and instability, anti-windup schemes must be implemented.

Fig. 7.9 illustrates the difference in system response when a fictitious saturation (implemented via a saturation switch and a predefined value in the constant block) is compared to a code-ignored saturation. The anti-windup scheme effectively addresses the saturation-related issues, ensuring stable and accurate system behavior.

From the displayed plot, it can be observed how the anti-windup code lines and the introduction of the feedforward action in the control design managed to constrain the effect of saturation and to provide even a better response to the simple control where the saturation effect was not considered.

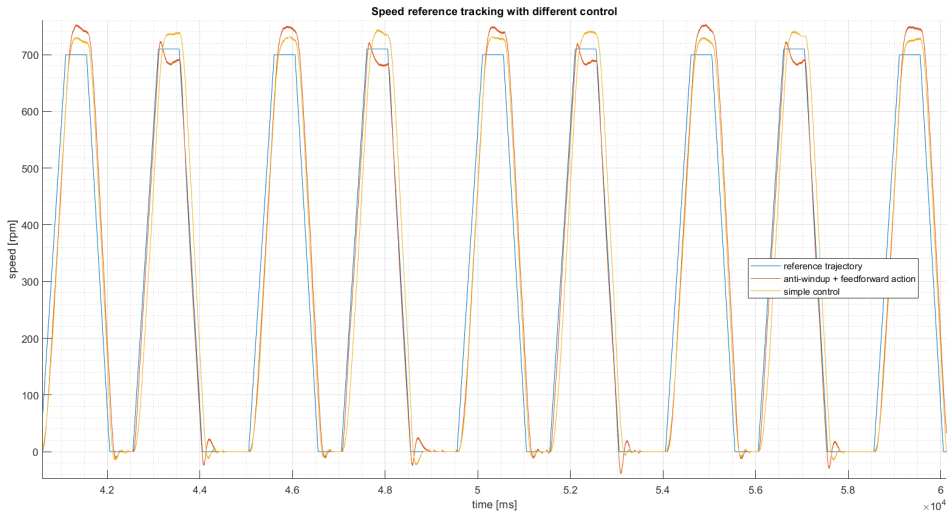

Figure 7.9: Effect of anti-windup and feedforward on tracking under saturation.

Chapter 8

Parameters Estimation (Adaptive Approach)

Till now we have used datasheet-parameters to model the motor and then tune our controller. However these data can be subject to small errors due to the endorsed tolerance range of the various parameters. In this regard a different control approach have been implemented, Adaptive control.

An adaptive regulator is able to modify *automatically* its own behavior in order to react to variations in the process dynamics and/or to external disturbances. They allow to:

- estimate online the value of the plant parameters
- adapt the control parameters on the basis of this estimation

Between the 2 general control scheme architecture (Model Reference Adaptive System and Self Tuning Regulators) it had been chosen to use the STR one. An essential component of a STR control scheme is the *parameter estimation algorithm* used to obtain the (unknown) parameters of the process. In this context the *Least Square (LS)* algorithm is a parameter estimation method commonly used in control system to estimate the parameters of a plant. The goal of LSE is to minimize the sum of weighted squared errors between the measured output and the predicted output of a model.

Two scripts have been used to estimate respectively L (armature inductance), R (armature resistance) and J (inertia), b (friction coefficent).

8.1 Estimation of L and R

From the run of the setup are recovered and used as input the voltage applied to the motor and as output the current response of the motor. Given the motor electric dynamic modeled as

$$V_a = L\frac{di_a}{dt} + Ri_a$$

rewriting in discrete time (discretizing using the backward difference method):

$$i_a(k+1) = \alpha_1 i_a(k) + \alpha_2 V_a(k)$$

where:

$$\alpha_1 = 1 - \frac{RT_s}{L}, \qquad \alpha_2 = \frac{T_s}{L}$$

and from this relationships:

$$L = \frac{\alpha_2}{\alpha_1} T_s, \qquad R = \frac{1-\alpha_1}{\alpha_2}.$$

The script estimates the parameters α_1 and α_2 by firstly defining the regressor matrix ϕ. For each sample k, the regressor vector is

$$\phi(k) = \begin{bmatrix} i_a(k-1) \\ V_a(k) \end{bmatrix}.$$

Then using the least square solution:

$$\alpha = (\phi^T \phi)^{-1} \phi^T y$$

or equivalently using the pseudo-inverse:

$$\alpha = \text{pinv}(\phi)\, y$$

where y is the current output vector. Finally from the estimated α_1 and α_2 we can calculate the value of R and L from the previous introduced formulas.

Furthermore the script uses a subset of the data for estimation corresponding to the first $N = 1000$ values. Different runs were made with different input (voltage supply values).

8.1.1 Results at 8 V and 24 V

In Fig. 8.1, the plot compares the model and the real system for two cases: using datasheet parameters (top plot) and using estimated parameters (bottom plot).

In the top plot, the model (blue line) and the real system (red line) show noticeable discrepancies, particularly in the transient regions where the current changes rapidly. This indicates that the datasheet parameters result in a model that does not accurately reflect the measured data. This is expected, as datasheet values are typically obtained under idealized conditions, which differ from the actual experimental setup (e.g., 24 V applied versus the rated 24 V).

In the bottom plot, the model generated using estimated parameters closely aligns with the real system. Both the transient and steady-state behaviors are nearly identical to the measured data. This improved match suggests that the least squares estimation effectively captures the system dynamics, accounting for variations that are not reflected in the datasheet values.

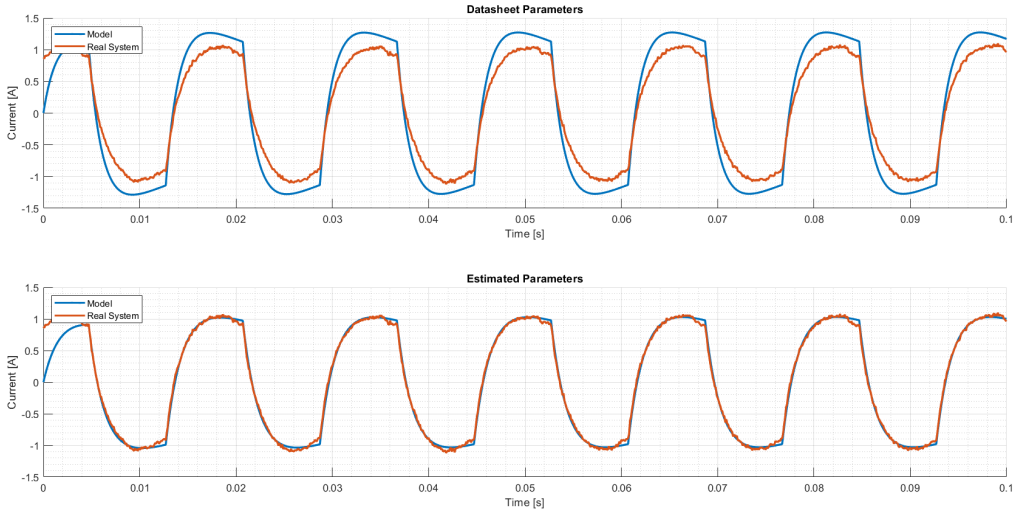

Figure 8.1: Comparison between estimated and datasheet parameters given 8 V as input.

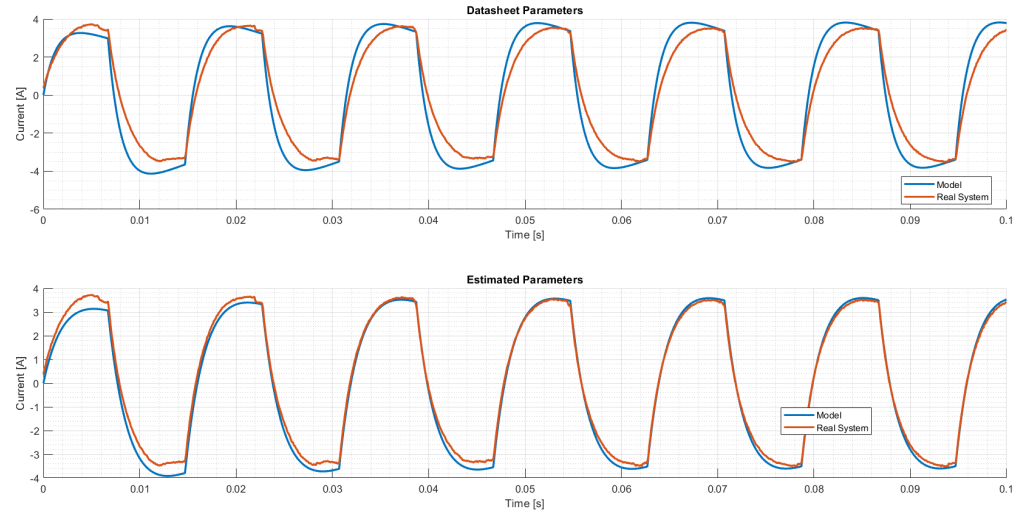

Figure 8.2: Comparison between estimated and datasheet parameters given 24 V as input.

Even at rated condition, the estimated parameter perform better than the datasheet one, indeed it can be seen the better accuracy of the bottom plot to track the measure current values. Nonetheless the datasheet parameters show an improved modeling compared to the low voltage one.

From the code it has been printed out also the value of these two estimated parameters. Given an input of 24 V the estimated values were:

$$R = 3.1908\ \Omega, \qquad L = 0.0064\ \text{H}$$

while in initial run with 8 V supply the parameters were:

$$R = 3.8189\ \Omega, \qquad L = 0.0061\ \text{H}$$

which underline again the higher difference between the 2 values at non-rated condition.

8.1.2 Effect of Data Length (N)

Not all collected data had been used for this estimation, indeed the portion of samples used was fixed at $N = 1000$, since it allows good performances of the algorithm and demand a light computational power. Increasing the number of addressed samples in some cases can lead to a better approximation of the parameters however given the periodicity of the input signal in this case no additional information can be retrieved from a bigger set of measured data. On the other hand an insufficient amount of data can lead to an incorrect estimation.

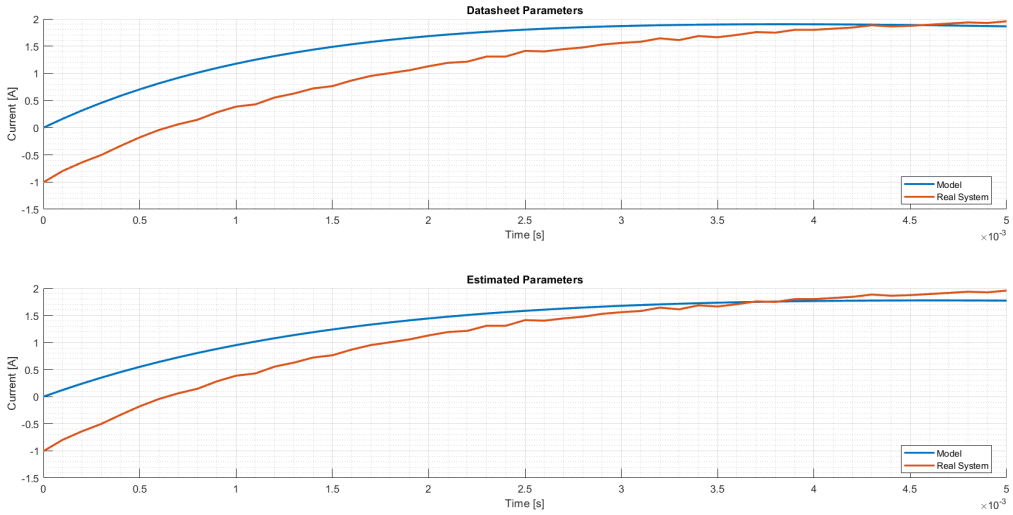

Figure 8.3: 50-sample estimation.

In Fig. 8.3 is shown the tracking of the model made with $N = 50$. In this case the values of armature resistance and inductance were approximated by the algorithm to:

$$R = 3.3695\ \Omega, \qquad L = 0.0055\ \text{H}.$$

A better view of the reason for this error is displayed by Fig. 8.4.

It can be seen how only a portion of the entire period of the measured current is used.

8.2 Estimation of J and b

For the estimation of these 2 parameters, the input data was the toque calculated from the measured current, while the output was the angular speed measured by

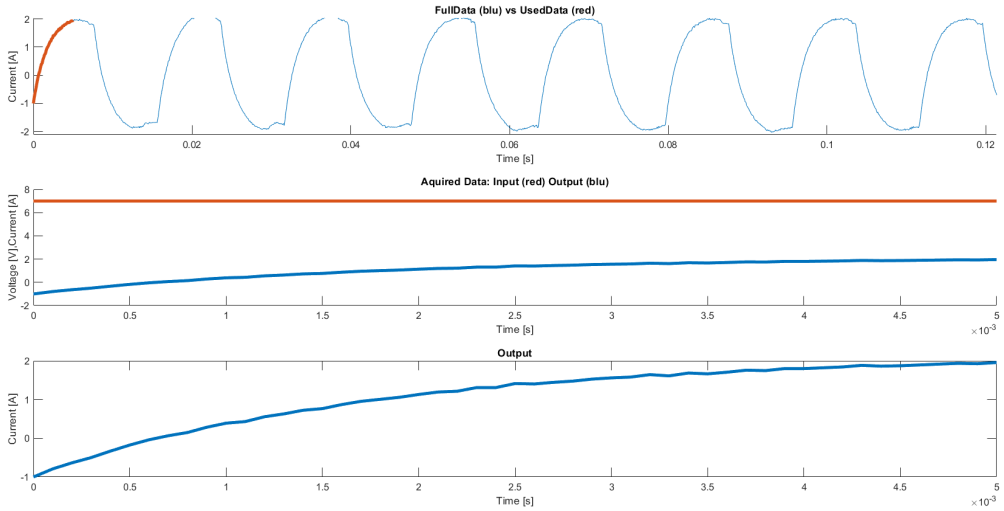

Figure 8.4: Used vs full data display.

the encoder. The mechanical equation for a DC motor is:

$$T_m = J\frac{d\omega_m}{dt} + b\omega_m + T_i$$

where $T_m = K_m i_a$ which accounts for the motor torque, $\frac{d\omega_m}{dt}$ corresponds to the angular acceleration and T_i is the load torque. After discretizing we get:

$$\omega_m(k+1) = \alpha_1\omega_m(k) + \alpha_2 T_m(k)$$

where:

$$\alpha_1 = 1 - \frac{bT_s}{J}, \qquad \alpha_2 = \frac{T_s}{J}.$$

The Matlab script estimates the parameters α_1 and α_2 by defining a regressor matrix ϕ:

$$\phi(k) = \begin{bmatrix} \omega_m(k-1) \\ T_m(k) \end{bmatrix}.$$

Given the following relationship between the input and outputs:

$$\omega_m(k) = \phi(k)^T\alpha$$

it has been used the least squares solution:

$$\alpha = (\phi^T\phi)^{-1}\phi^T y.$$

In conclusion the estimated value of J and b are derived according to:

$$J = \frac{\alpha_2}{\alpha_1}T_s, \qquad b = \frac{1-\alpha_1}{\alpha_2}.$$

8.2.1 Square-Wave Excitation and Operating Points

As for the L and R parameter, multiple runs under different conditions where made. Given a reference of a square-wave of 50% duty cycle.

In the low voltage mode (8 V) (Fig. 8.5) we can already see how the estimated parameters perform way better than the datasheet one, which seems very off track. The reason for than is mainly due to the friction coefficient which was set to null. The calculated parameters under these condition provide a fluctuating response in terms of speed (bottom plot).

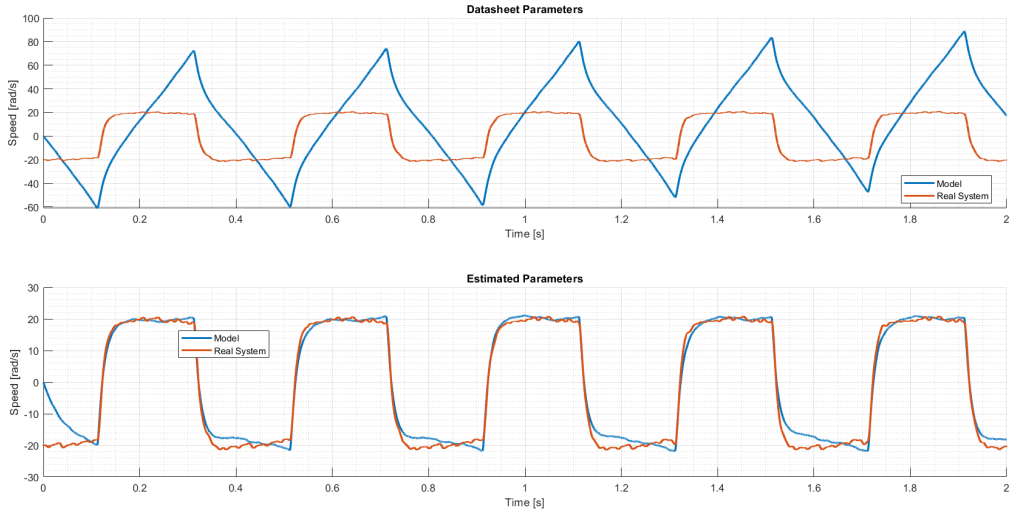

Figure 8.5: Comparison between estimated and datasheet parameters given 8 V as input.

While, when supplied with rated voltage (Fig. 8.6), their behavior is more clean and aligns better with the reference trajectory (square wave) and measured data. For what concerns the datasheet parameter model, it provides a better output with the rated voltage supply rather than the low voltage mode but still is not reliable.

Other runs were made with different periods of the reference trajectory, the plots seen up to this point were retrieved from a tracking of a 0.4 s period of a square waveform, if this period is increased to 0.8 s the graph in Fig. 8.7 is generated.

This phenomenon occurs because, with a shorter period (higher frequency), the square wave excites the higher-frequency dynamics of the system. These dynamics are critical for accurately estimating parameters such as inertia and friction. Conversely, with a longer period, the system spends more time in steady-state conditions, reducing variability in the data and diminishing the accuracy of the least-squares estimation, which primarily relies on dynamic transitions.

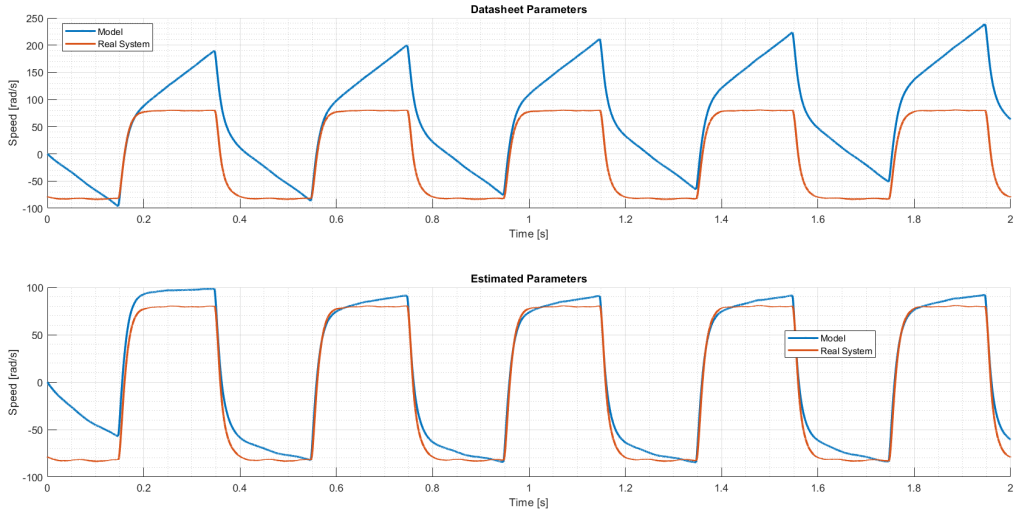

Figure 8.6: Comparison between estimated and datasheet parameters given 24 V as input.

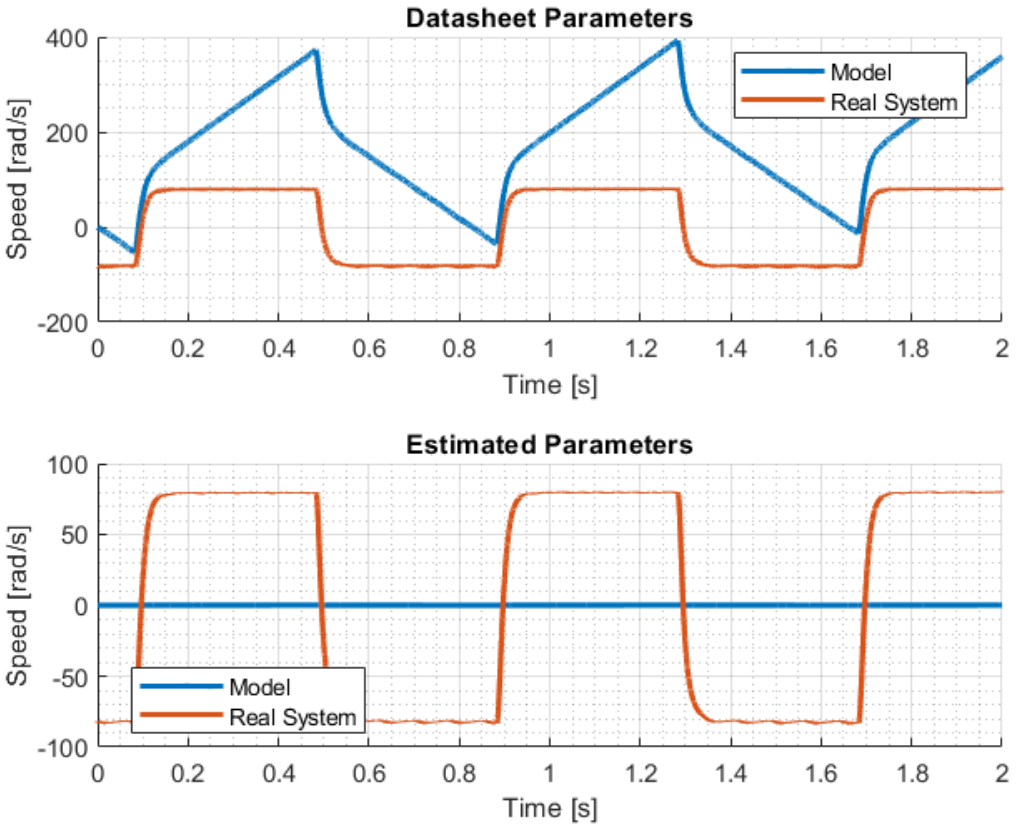

Figure 8.7: Estimation performed on a square wave with period 0.8 s.

8.2.2 Voltage Dependence of b

The friction coefficient (b) is dependent on the supply voltage. To illustrate this relationship, a plot interpolating the estimated values of b using a fifth-degree polynomial function is displayed in Fig. 8.8.

It is evident that at higher voltages, this coefficient becomes smaller. This is because the motor has more power to overcome the resistive friction torque more quickly and efficiently. Since the voltage supplied to the motor is directly related to the motor's speed, it can be concluded that the friction coefficient is also dependent on the motor's speed.

The estimation of the b parameter lies in the range of $[1.8 - 0.1] \times 10^{-3}$.

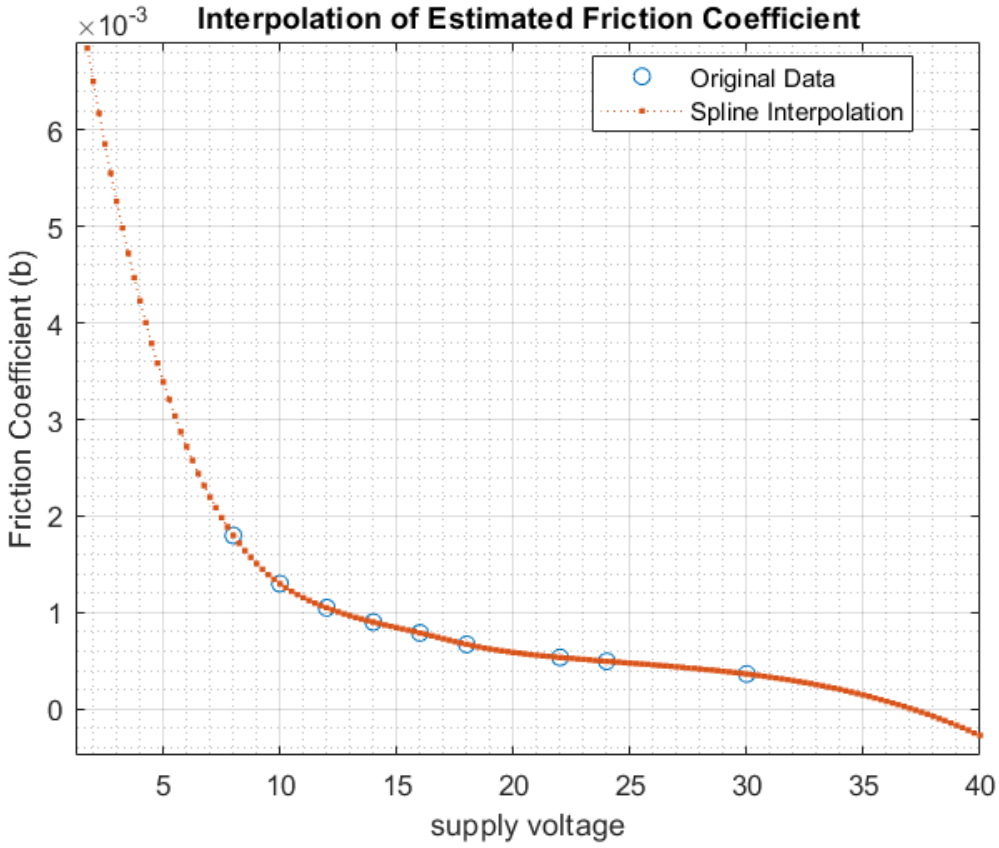

Figure 8.8: Interpolation of estimated b values.

Important to note that all the estimation where made in absence of any kind of load.

Chapter 9

Deadbeat and Dahlin Controllers

Up to this point the only controller configuration seen is the cascade PI control design, however a controller can be designed according to different methodologies, among them the *Deadbeat* method. The deadbeat design imposes some desired properties of the output signal if the input reference is a classical function (step). In this case, typical deadbeat specifications are:

- the output must reach its final value in the minimum possible time
- the steady state error must be zero
- no oscillation must be present between samples

9.1 Deadbeat Controller

Assuming that the system model is stable and minimum phase (assumptions that hold in this case) the controller $D(z)$ has a simple expression:

$$D(z) = \frac{1}{G_p(z)} \frac{z^{-k}}{1 - z^{-k}} = \frac{1}{G_p(z)} \frac{1}{z^k - 1}$$

where $k > r_p$ being r_p the relative degree of the plant transfer function.

9.1.1 Simulink Implementation

It is worth noting that the controller can be implemented as a simple transfer function with a saturation block in cascade. Alternatively, an anti-windup scheme can be adopted to prevent issues related to infeasible control actions.

9.1.2 Anti-Windup Architecture

As an alternative to the previously coded-handled saturation, a design architecture has been implemented. This anti-windup scheme is a specific case of the framework

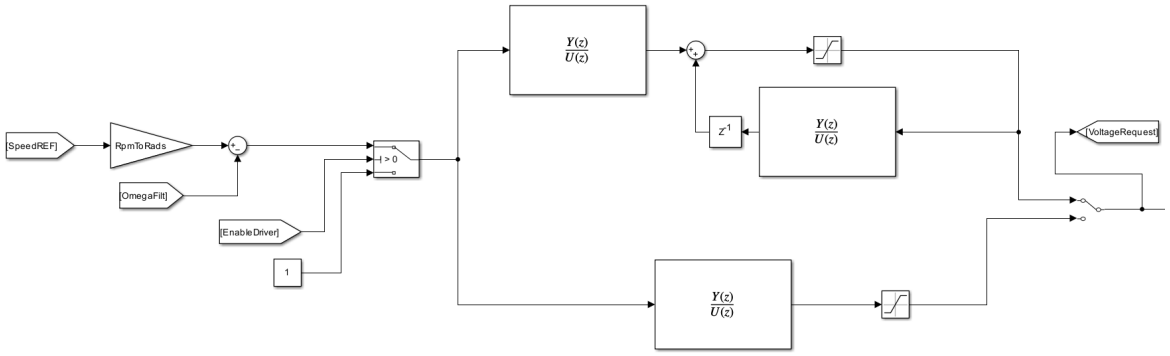

Figure 9.1: Implementation of the deadbeat controller in the Simulink environment.

illustrated in Fig. 9.2, where $f(z)$ represents an asymptotically stable polynomial such that:

$$\deg\{F(z)\} \geq \deg\{B(z)\}, \qquad \frac{B(1)}{F(1)} \geq 0.$$

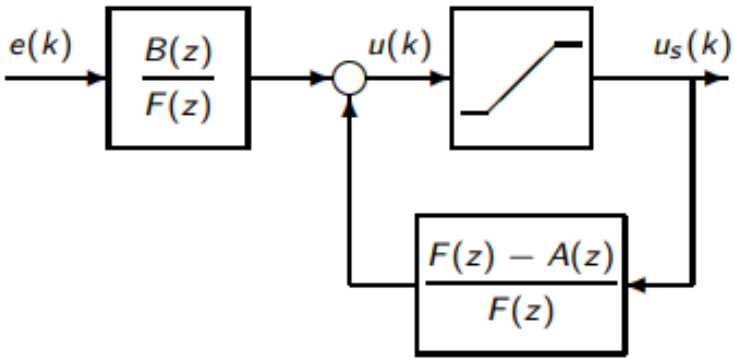

Figure 9.2: Anti-windup scheme.

This architecture can be implemented with the datasheet values as well as with the estimated parameters (Fig. 9.3).

The plot displays the runs performed with the motor model alongside those obtained from the real device. It is evident that the controller achieves a faster steady-state value (in less than 1 ms) and eliminates the transient error, all of which are characteristic of the deadbeat controller. An interesting observation from this graph is that, in both cases (controllers based on estimated and datasheet parameters), the response is slightly faster in the setup compared to the model used for simulation. However, the transient patterns are almost identical across all four cases (no oscillations). This slight discrepancy might be due to the step angular speed requested, which is 1000 rpm, a value lower than the motor's rated speed.

When saturation is considered, and no anti-windup loop is implemented (altering the switch in the Simulink block scheme), different results can be observed, as shown in Fig. 9.4.

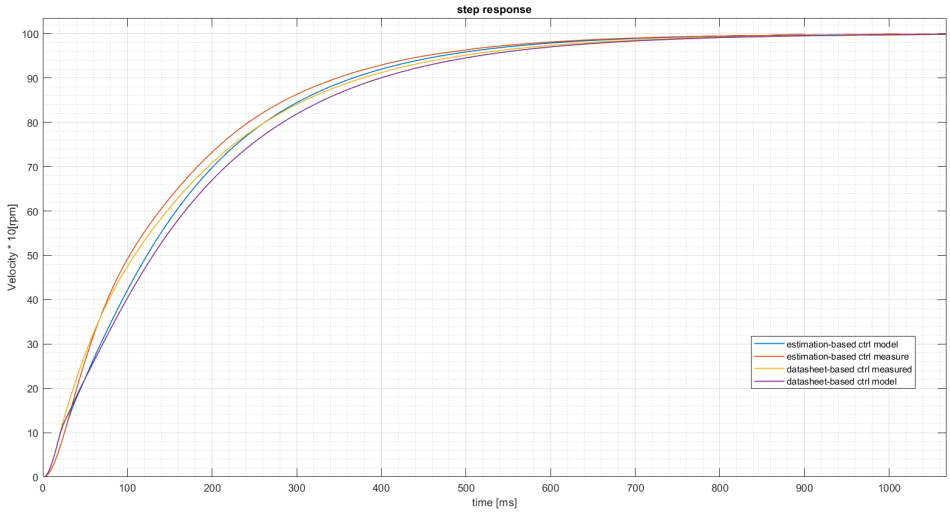

Figure 9.3: Deadbeat controller with anti-windup: step responses for model and setup.

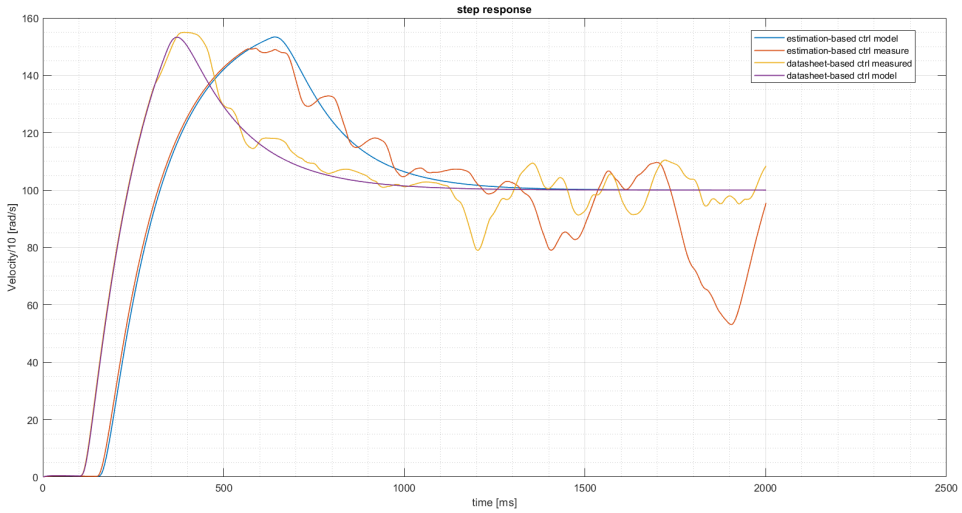

Figure 9.4: Deadbeat controller with saturation but no anti-windup loop.

The model simulation produces a smoother response compared to the noisy output from the setup, regardless of whether datasheet or estimated parameters are used. While this result might have been expected, it highlights an inherent limitation: simulations must closely match the real-world outcome to predict device behavior accurately. Even when the real system performs better than the simulated one, adjustments to the model are necessary to align it with observed results, even more when the real-world performance is worse.

9.2 Dahlin Controller

If the reference trajectory is not a simple/classical step a different kind of adaptive control must be implemented to obtain better performances. In this regard it was introduced the *Dahlin* control method. This approach defines a desired closed-loop response by choosing a reference model for the system's behavior. This desired transfer function can be written in the most explicit form as:

$$G_{mz}(z) = \frac{\left(1 - e^{-\frac{T_{\text{Sample}}}{\lambda}}\right) z^{-(N+1)}}{1 - \left(1 - e^{-\frac{T_{\text{Sample}}}{\lambda}}\right) z^{-1}}$$

where λ represents the desired time constant for the closed-loop system.

9.2.1 Controller Expression and Implementation

Given that the actual plant is modeled as:

$$G_p(z) = \frac{B(z)}{A(z)}$$

the Dahlin controller is derived as:

$$D(z) = \frac{1}{G_p(z)} \frac{G_m(z)}{1 - G_m(z)}.$$

The Simulink scheme block is the same used for the deadbeat controller, thus also in this case is present the switch allowing to integrate an anti-windup loop to avoid saturation problems. After some runs in the model environment as well as the real setup a good value of λ has been considered to be $\lambda = 0.022$.

9.2.2 Step-Response Results

Evaluating for a step input reference the result on Fig. 9.5 have been printed.

It can immediately be seen the slowest response compared to the deadbeat method (it reaches the steady-state value after almost the double of the time) but it was expected since the Dahlin method aim to a smoother output rather than a fast one. Again the model curve in absence of an anti-windup scheme is much cleaner than the real one, even though the last one is much better than the deadbeat corresponded. Furthermore when saturation takes place and no feedforward action take place we can observe a delay on the answer of the system.

9.2.3 Tracking of Complex Trajectories

If the reference trajectory has a more complex pattern (Fig. 9.6) the Dahlin controller can be very effective.

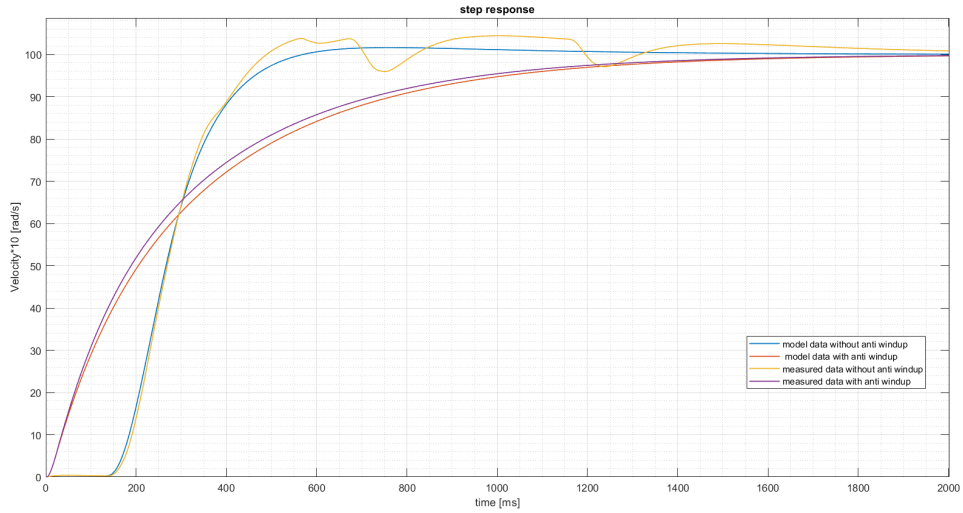

Figure 9.5: Responses to a step input with Dahlin controller.

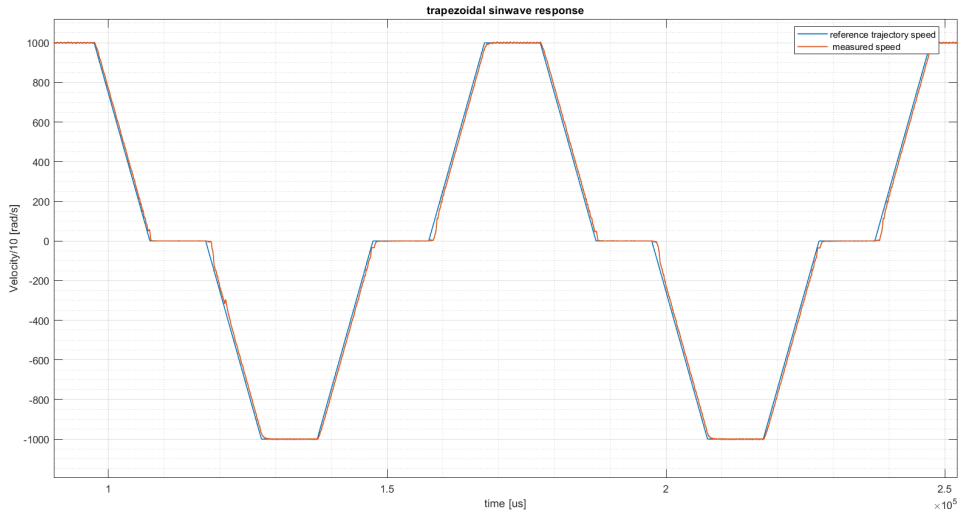

Figure 9.6: Response to a trapezoidal sinwave with Dahlin controller.

Even though some delays are present in the initial phase of a transient, it can still be considered a good tracking.

9.3 Effect of Online Disturbance: Braking Resistor Connection

In this particular run, some changes were carried out during the recovery of measured data, indeed the brake resistor where connected while the motor was running with the idea to see what changes could have been witnessed in the response of the system. Besides a little spark on the measured speed (not even easily noticeable) (Fig. 9.7) the

tracking resulted to be unaffected.

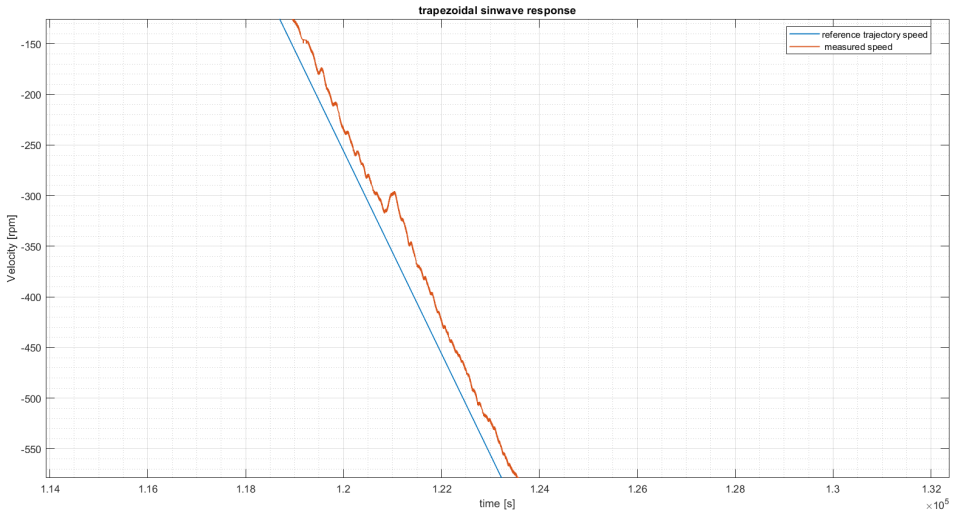

Figure 9.7: Spark on the speed measure (transient during brake-resistor connection).

This impulse on the speed was expected since as soon as the braking resistor is connected the resistance to the motoring force of the shaft have increased resulting on a little drop of the overall velocity. Furthermore an increase on the required voltage to ensure the tracking was expected and verified by the measured data (Fig. 9.8).

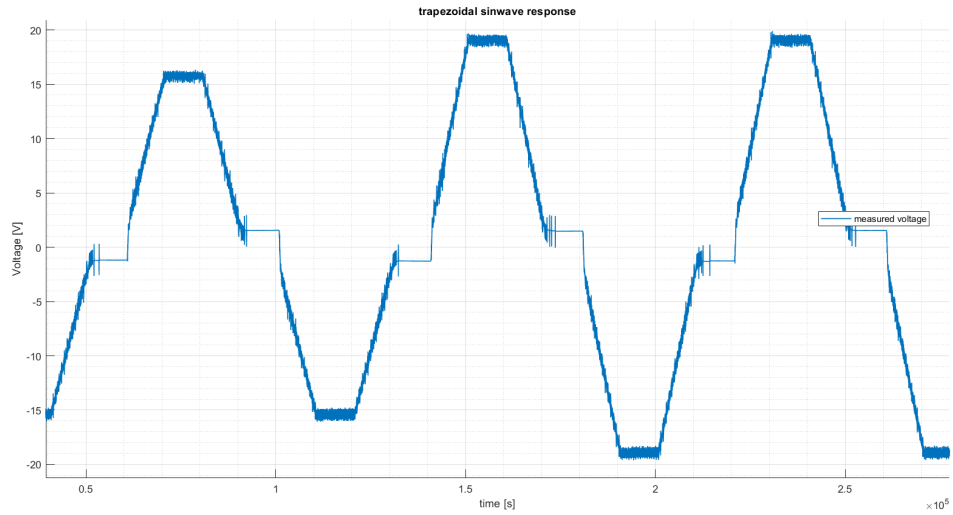

Figure 9.8: Voltage measured during trapezoidal sinwave tracking.

The change affected also the measured current as it can be seen from Fig. 9.9.

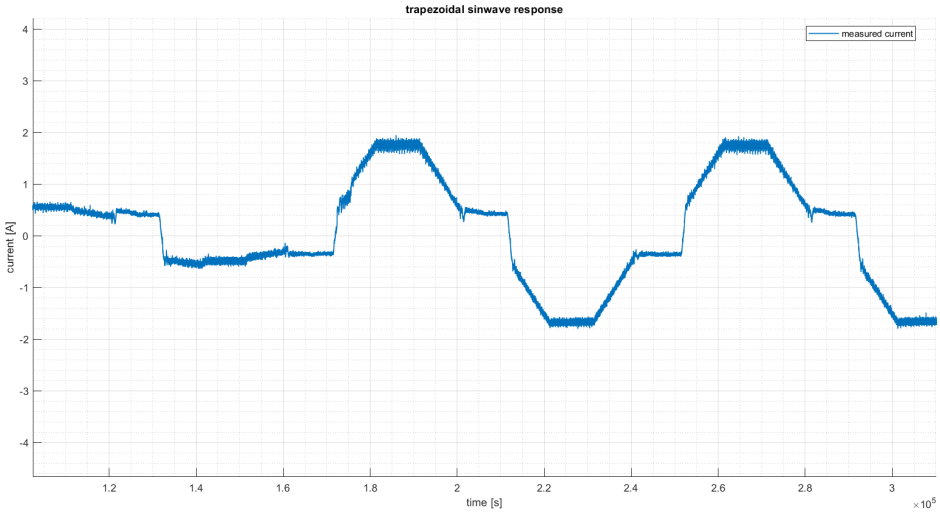

Figure 9.9: Current measured during trapezoidal sinwave tracking.